

zkRNN: Zero-Knowledge Proofs for Recurrent Neural Network Inference

Fatemeh Zarinjoui
Sharif University of Technology
Tehran, Iran
ftm.zariin@gmail.com

Behzad Abdolmaleki
University of Sheffield
Sheffield, United Kingdom
behzad.abdolmaleki@sheffield.ac.uk

Maryam Zarezadeh
Barkhausen Institut
Dresden, Germany
maryam.zarezadeh@barkhauseninstitut.org

Bhavish Mohee
University of Sheffield
Sheffield, United Kingdom
bhavishmohee@gmail.com

Aysajan Abidin
COSIC, KU Leuven
Leuven, Belgium
aysajan@kuleuven.be

Stefan Köpsell
Barkhausen Institut
Dresden, Germany
stefan.koepsell@barkhauseninstitut.org

Abstract—Neural networks have achieved remarkable success across a wide range of domains, including applications involving sequential data such as natural language processing and time-series prediction. However, in many real-world deployments, it is essential to ensure the integrity of the inference process—namely, that the output of a model is correctly computed—without revealing the model’s data. While prior work has introduced zero-knowledge proof (ZKP) schemes for convolutional and feedforward neural networks, these do not extend naturally to recurrent architectures due to the challenges introduced by temporal dependencies and weight sharing.

In this paper, we propose zkRNN, a novel ZKP system for recurrent neural networks (RNNs), enabling the prover to demonstrate that a model’s output is correctly computed over a sequential input without revealing any information about the model parameters. Our approach builds upon the GKR protocol, and a recursive sum-check framework introduced in prior work and adapts them to handle the recurrent structure of RNNs. We design a circuit representation that encodes hidden-state transitions, unrolls computation across time steps, and shares weights in a manner compatible with sum-check-based verification. Our protocol achieves polylogarithmic verifier time and proof size in the size of the final iteration circuit and remains independent of the sequence length. The evaluation results demonstrate practical proof generation and succinct, sequence-length-independent verification, with second-scale proving and millisecond-scale verification.

Index Terms—Zero-knowledge proof, Recurrent neural network, Verifiable inference, Machine learning, RNN

I. INTRODUCTION

As machine learning (ML) systems become more common in privacy-sensitive areas like healthcare, finance, government, and secure communications, ensuring accurate model predictions is a major challenge [1], [2]. In many real-world situations, the entity making predictions, such as a cloud service, may not be trusted [3], [4]. Meanwhile, the person requesting the prediction often cannot share sensitive information. Therefore, the ability to verify model predictions without exposing the input, output, or model details is essential for trustworthy ML [5]. Zero-knowledge proof (ZKP) provides a solid cryptographic basis for this verification. A ZKP is

a cryptographic protocol that allows a prover to convince a verifier that a statement is true, while revealing no information beyond the validity of the statement [6]. In recent years, the ML community has increasingly used zero-knowledge (ZK) and succinct proof systems to offer verifiable computation guarantees for inference and training tasks. These methods allow auditors or users to check correctness without having to redo costly neural computations, paving the way for verifiable machine learning (VML) [7]–[9].

Recent studies have shown how to create efficient ZKP proofs for inference on feedforward and convolutional neural networks (CNNs) [3], [10], [11]. These systems usually rely on the Goldwasser-Kalai-Rothblum (GKR) protocol [12] and its variations, which verify computations represented as layered arithmetic circuits using interactive proof and sumcheck techniques. By transforming each layer of a neural network into a circuit over a finite field, these methods can efficiently prove correct inference, even for deep networks with millions of parameters. However, these approaches fundamentally assume acyclic computation graphs, meaning networks where each layer depends only on the previous one with no information flowing backward.

Recurrent Neural Networks (RNNs), on the other hand, introduce feedback loops via their hidden states. At each time step, an RNN takes in an input and updates its hidden state based on the new input and the previous hidden state. This sequential dependence makes RNNs suitable for modeling time-varying or ordered data. They are a key component of sequence modeling and are used in fields like natural language processing, time-series forecasting, and control systems [13], [14]. Despite their importance, RNNs present significant challenges for ZK frameworks. The iterative nature of an RNN means the computation graph is no longer static or acyclic: weights are reused across time steps, and hidden states change dynamically. Existing ZK systems struggle to manage these recurrent links efficiently. Unrolling the network for all time steps results in a circuit that grows exponentially, negating succinctness and efficiency. This disconnect between cryp-

tographic verifiability and sequential learning leads to the question: Can we create a ZKP system that verifies RNN inference while remaining asymptotically efficient and zero-knowledge? In particular, the verifier’s time and proof size should not grow linearly with the sequence length, and the proof should reveal no information about the model parameters, intermediate hidden states, or private inputs. Solving this problem requires rethinking how recurrent computations are represented within arithmetic circuits and how proofs can be composed across dependent time steps.

In this work, we present zkRNN, a ZKP framework for verifiable inference over RNNs. zkRNN bridges the gap between ZK theory and sequential ML by creating a circuit representation that captures hidden state transitions, unrolled computation across time steps, and shared weights in a way that works with sumcheck-based verification [12]. Our approach builds on recursive proof techniques and sumcheck-inspired constructions, but introduces key innovations that make ZK verification of RNNs both feasible and scalable. Specifically, zkRNN features a recursive architecture that treats each RNN time step as an independently provable component. This modular design allows verification of each step’s computation—made up of linear transformations and non-linear activations—using efficient sumcheck protocols, while keeping steps consistent through commitments to the changing hidden state. We also develop a generalized arithmetic circuit representation that captures shared weights and temporal dependencies, and adjust recursive proof composition techniques to combine per-step proofs logarithmically in relation to the sequence length.

Beyond the main proof system, zkRNN includes a streaming verification mode that meets the real-time needs of many sequence-based applications. Instead of a “compute-then-prove” approach, zkRNN provides inference outputs right as inputs arrive, while generating and recursively combining proofs in the background. This “prove-as-you-go” model significantly lowers latency and makes verifiable inference practical for online translation, event prediction, and secure time-series analysis. It should be mentioned that zkRNN provides privacy guarantees only for *inference*. Specifically, it hides the model weights, the hidden-state values, and optionally the inference outputs, ensuring that none of these are revealed in the ZKP. However, zkRNN does not protect the training data or any information arising during training (e.g., gradients or optimization updates). Zero-knowledge protection for training data is the goal of ZK training protocols, whereas this work focuses solely on ZKPs of inference for recurrent neural networks.

A. Our Contributions

zkRNN introduces a novel, end-to-end framework for verifiable and privacy-preserving inference over recurrent architectures. It preserves the succinctness and zero-knowledge properties of ZK systems while extending their applicability to models with temporal structure and stateful computation. Our main contributions are summarized as follows:

- *Zero-knowledge inference for RNNs.* zkRNN provides a ZKP system that natively models forward recurrence and shared weights within an arithmetic-circuit representation compatible with sumcheck-based verification.
- *Integrated recursive verification of sequential computations.* Within the zkRNN framework, we extend recursive sumcheck techniques to verify hidden-state transitions and unrolled recurrent computation across time, yielding verifier complexity and proof size that scale poly-logarithmically rather than linearly with sequence length.
- *Streaming, low-latency proof architecture.* zkRNN incorporates a streaming execution mode that outputs inference results in real time while proofs are generated and aggregated concurrently, thereby minimizing end-to-end latency.
- *Prototype implementation and empirical validation.* We implement zkRNN and demonstrate that this single framework achieves efficient proof generation and succinct verification across realistic RNN workloads; for models with hidden size up to $h = 256$ and sequence length up to $T = 2^7 = 128$, proof generation takes seconds, while verification completes in tens of milliseconds with proof sizes below a few hundred kilobytes, empirically validating succinct and sequence-length-independent verification for sequential inference.

By enabling ZKPs for RNN inference, zkRNN substantially broadens the scope of VML to tasks involving time-series data, natural language, and sequential decision-making. Our framework maintains privacy, ensures soundness, and achieves practical efficiency across long input sequences, marking a significant step toward end-to-end VML for sequential neural models.

B. Technical Overview

In this section, we provide an overview of zkRNN, our ZKP framework for verifying RNN inference. Our design tackles the main challenges of applying ZKP methods, typically designed for feedforward and convolutional networks, to the sequential and stateful computations of RNNs. The framework combines optimized sumcheck protocols, GKR-style proofs [12], polynomial commitment methods [15], and recursive proof composition to allow for efficient and scalable zero-knowledge verification across long input sequences.

a) Proof Construction at Each Time Step. Each RNN time step consists of two main computation stages: linear transformations (matrix-vector multiplications) and non-linear activations. We combine the linear operations, including weight multiplications and bias additions, into a single matrix-vector multiplication using an enhanced input vector and weight matrix. This approach allows us to apply Thaler’s time-optimal sumcheck protocol [16], resulting in lower prover complexity and considerable efficiency improvements compared to standard circuit representations.

b) GKR-Based Verification of Nonlinear Activations: For non-linear activations, such as ReLU and tanh, we use GKR-based proofs enhanced with the Dao–Thaler sumcheck technique [17]. Non-linear functions are represented as arithmetic circuits and verified using bit-decomposition and consistency

checks. This ensures correctness while minimizing overhead. We express binary constraints and sign checks as specific sumcheck instances, preventing circuit growth and aiding succinctness.

c) Recursive Proof Composition and Commitment Aggregation.: To enable efficient verification over long sequences, we recursively combine per-step proofs. This method reduces proof size from linear to constant in relation to the number of time steps. However, recursion brings challenges, especially for polynomial commitment openings.

We tackle these challenges with an aggregation scheme based on evaluation reduction and the Orion polynomial commitment method [15]. We merge several evaluation proofs into a single proof, verified only at the last recursion step, achieving an overhead complexity of $O(\log^2(h))$ time for the verifier, where h is the hidden state size. This approach maintains scalability without sacrificing zero-knowledge security. In particular, zkRNN maintains a prover time linear in the number of time steps, while the verifier and proof size remain polylogarithmic in the final iteration circuit size and independent of the sequence length.

d) Streaming Architecture for Real-Time Inference.: A core design principle of zkRNN is its compatibility with the streaming nature of real-world RNN tasks. Rather than adopting a monolithic "compute-then-prove" model, our framework operates concurrently: it returns the inference output for each time step immediately, while the corresponding zero-knowledge proof is generated and recursively folded in the background. This "prove-as-you-go" approach ensures that a final, succinct proof for the entire interaction sequence can be finalized with minimal delay, aligning the verification process with the low-latency demands of applications like real-time translation.

e) Efficiency and Soundness Enhancements.: We also boost performance and security with several techniques: *i) Evaluation reduction*: condenses multiple evaluation checks into a single sumcheck instance, thereby lowering verifier circuit size. *ii) Sumcheck-friendly Fiat-Shamir hashing*: utilizes ZK-friendly hash functions with low multiplicative complexity, such as MiMC [18], to enable efficient non-interactive transformation. *iii) Tree-based recursion*: ensures soundness across numerous time steps by verifying proofs in a logarithmic-depth tree structure.

So, zkRNN advances ZKP methods into sequential neural inference by integrating optimized sumcheck protocols, generalized GKR circuits, and recursive composition with commitment aggregation. This cohesive framework enables succinct, scalable, and privacy-preserving verification of RNN computations over long input sequences, significantly expanding the range of VML. The formal recursive framework for RNN inference proofs is presented in Appendix D.

C. Related Work

SafetyNets [3], is the first scheme to enable verifiable inference for DNNs using ZKPs. Designed for untrusted cloud servers, this approach takes advantage of the structural

similarity between the GKR protocol and multi-layer neural networks. By combining the GKR protocol with an interactive proof system for matrix multiplication [19], SafetyNets ensures end-to-end verifiability while significantly reducing bandwidth overhead. To address the challenge of nonlinear computations in neural networks, which are difficult to express in arithmetic circuits and process within the proof framework, SafetyNets restricts its support to quadratic activation functions and sum pooling operations. However, this limitation reduces the approach's flexibility. Although SafetyNets lacks zero-knowledge properties, privacy is not a concern in this setting since both the model and input are supplied by the verifier. vCNN [20] is a verifiable inference framework designed to accelerate proof generation for CNNs. It employs zk-SNARK proof protocols, utilizing the quadratic polynomial program [21] for convolutional layers and the quadratic arithmetic program [22] for non-convolutional components. To verify model integrity, vCNN integrates the commit-and-prove SNARK [23] technique to establish a linking proof between these parts. While vCNN efficiently verifies small-scale CNN models, its verification time and storage demands increase significantly as the model size grows. Mystique [24] addresses the lack of efficient ZKP protocols for verifying inference results in neural networks. Built on the subfield vector oblivious linear evaluation (VOLE) interactive proof protocol [25], it introduces three key advancements. First, it provides optimized ZKP building blocks that simplify the development of zero-knowledge ML systems by abstracting cryptographic complexities. Second, it proposes three specialized ZKP protocols: one for efficient conversion between arithmetic and Boolean values, another for integrating public commitments with private authenticated values, and a third for reconciling fixed-point and floating-point representations used in cryptographic and machine learning computations. Finally, Mystique optimizes ZKP-based matrix multiplication, achieving sublinear private multiplications, significantly improving efficiency in matrix-related operations. Le et al. [26] propose a ZKP framework for efficient verifiable inference that integrates VOLE [27] with range, exponent, and lookup proofs to reduce the cost of non-linear layers, enabling scalable verification across multiple neural network architectures, including CNNs. Hao et al. [28] study efficient ZK verification of non-linear functions in ML models, introducing table-lookup techniques to replace costly Boolean circuits and digital decomposition to control lookup table size, significantly improving scalability and efficiency.

Concurrent Work. Independent and concurrent with our work, Li and Fan propose SUMMER [29], a recursive ZKP system for verifiable training of RNNs. SUMMER focuses on proving the correctness of backpropagation through time, including forward passes, gradient computation, and parameter updates, and achieves iteration-succinct proofs via recursive composition. In contrast, our work addresses ZK proofs of RNN inference, targeting the correctness of forward evaluation over sequential inputs while hiding model parameters and intermediate states. Conceptually, SUMMER proves how an

RNN is trained, while zkRNN proves how a trained RNN is evaluated. While both systems utilize sumcheck-based techniques and recursion to manage temporal dependencies and weight sharing, they operate under different problem settings—training versus inference—and make distinct design choices for handling non-linearities, commitments, and streaming execution.

II. PRELIMINARIES

A. Notation and algebraic preliminaries

Notation. Throughout this paper, λ denotes the security parameter and $\text{negl}(\cdot)$ a negligible function. All arithmetic is over a finite field $\mathbb{F}$. “PPT” stands for probabilistic polynomial time. For $m \in \mathbb{N}$ we write $[m] := \{0, 1, \dots, m-1\}$. We use bold lowercase letters (e.g., $\mathbf{v}, \mathbf{w}$) for vectors and bold uppercase letters (e.g., $\mathbf{A}, \mathbf{B}$) for matrices; for $\mathbf{v} \in \mathbb{F}^\ell$ we write $\mathbf{v} = (v_1, \dots, v_\ell)$. We reserve x_t (or x_i) for the RNN input at time step t .

Multilinear Extensions. Let $V : \{0, 1\}^\ell \rightarrow \mathbb{F}$ be a function defined over the Boolean hypercube. The *multilinear extension* $\tilde{V} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ is the unique multilinear polynomial that agrees with V on all Boolean inputs; that is, for all $\mathbf{b} \in \{0, 1\}^\ell$, we have $\tilde{V}(\mathbf{b}) = V(\mathbf{b})$.

This extension can be written explicitly as: $\tilde{V}(\mathbf{x}) = \sum_{\mathbf{b} \in \{0, 1\}^\ell} V(\mathbf{b}) \cdot \beta_{\mathbf{b}}(\mathbf{x})$, where $\beta_{\mathbf{b}} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ is the multilinear Lagrange basis polynomial associated with the point $\mathbf{b}$, given by:

$$\beta_{\mathbf{b}}(\mathbf{x}) = \prod_{i=1}^{\ell} ((1 - x_i)(1 - b_i) + x_i b_i).$$

In practice, many objects such as vectors and matrices can be viewed as functions over the Boolean hypercube and naturally extended in this way. Specifically:

- A vector $\mathbf{v} = (v_0, v_1, \dots, v_{n-1}) \in \mathbb{F}^n$, where $n = 2^k$, can be interpreted as a function $v : \{0, 1\}^k \rightarrow \mathbb{F}$ mapping the binary representation of index i to v_i . Its multilinear extension $\tilde{v}$ is then defined over $\mathbb{F}^k$.
- A matrix $A \in \mathbb{F}^{n \times m}$, where $n = 2^k$ and $m = 2^l$, can be interpreted as a function $A : \{0, 1\}^{k+l} \rightarrow \mathbb{F}$, and its multilinear extension $\tilde{A}$ is defined over $\mathbb{F}^{k+l}$.

B. Cryptographic tools

Incrementally Verifiable Computation (IVC). IVC, introduced by Valiant [30], enables the construction of succinct proofs for iterative computations, where each step builds upon the previous one. Let F be a deterministic function modeling one step of a computation, and let $\omega_0, \dots, \omega_{i-1}$ be auxiliary inputs (or witnesses) used at each step. Define a sequence of outputs $\{z_k\}_{k=0}^i$ via:

$$\forall k \in [i] : z_{k+1} = F(z_k, \omega_k).$$

An IVC scheme allows a prover to incrementally produce a proof π_i attesting that there exists a sequence of witnesses $\omega_0, \dots, \omega_{i-1}$ such that z_i was computed correctly from z_0 through i applications of F . Importantly, the cost of verification is independent of the number of iterations i , and the

prover maintains only a small memory footprint. Formally, an IVC scheme consists of the following three algorithms:

- $G(1^\lambda) \rightarrow \text{pp}$: On input the security parameter λ , outputs public parameters pp .
- $P(i, z_0, z_{i-1}, \omega_{i-1}, \pi_{i-1}, \text{pp}) \rightarrow (z_i, \pi_i)$: On input the iteration index i , initial value z_0 , previous output z_{i-1} , auxiliary input ω_{i-1} , previous proof π_{i-1} , and public parameters pp , outputs the new output z_i and a new proof π_i .
- $V(i, z_0, z_i, \pi_i, \text{pp}) \rightarrow \{\text{accept}, \text{reject}\}$: On input the iteration index i , initial value z_0 , current output z_i , proof π_i , and public parameters pp , the verifier accepts if and only if the proof is valid.

Each proof π_i is a (zero-knowledge) argument for the relation:

$$\mathcal{R}_i^{\text{IVC}} = \{((i, z_0, z_i), (\omega_0, \dots, \omega_{i-1})) \mid \forall k \in [i] : z_{k+1} = F(z_k, \omega_k)\}.$$

In this paper, we used the method for constructing IVC via recursive proof composition. Given a succinct non-interactive argument system (P_b, V_b) for a baseline relation, we define an augmented function F_A as:

$$F_A(i+1, z_0, z_i, \omega_i, \pi_i) = (F(z_i, \omega_i), V_b(i, z_0, z_i, \pi_i)).$$

The prover computes $z_{i+1} = F(z_i, \omega_i)$ and invokes P_b to generate a new proof π_{i+1} certifying that F_A evaluated correctly. This construction ensures that the size of the proof and the cost of verification remain constant across iterations, while enabling verification of the full computation trace up to step $i+1$.

Polynomial Commitment Scheme (PCS). PCS enables a prover to commit to a polynomial and later prove the evaluation of the committed polynomial at any chosen point, without revealing the full polynomial. This is a core primitive in many modern zero-knowledge proof systems that require efficient verification of polynomial relations. A PCS consists of the following algorithms:

- $\text{KeyGen}(1^\lambda, \ell, d) \rightarrow \text{pp}$: On input a security parameter λ , number of variables ℓ , and degree bound d , this algorithm outputs public parameters pp .
- $\text{Commit}(f, r, \text{pp}) \rightarrow \sigma$: On input a polynomial f of total degree at most d , randomness r , and public parameters pp , the prover outputs a commitment σ to f .
- $\text{Open}(f, r, x, \text{pp}) \rightarrow (y, \pi)$: On input a point $x \in \mathbb{F}^\ell$, polynomial f , randomness r , and public parameters pp , the prover computes $y = f(x)$ and an evaluation proof π .
- $\text{Verify}(\sigma, x, y, \pi, \text{pp}) \rightarrow \{\text{accept}, \text{reject}\}$: The verifier checks whether y is the correct evaluation of the committed polynomial at point x using the proof π .

A secure PCS must satisfy the following properties:

- **Completeness:** For any correctly generated commitment and evaluation proof, the verifier accepts with probability 1.
- **Evaluation Binding (Knowledge Soundness):** It is infeasible for a malicious prover to produce a valid proof for a different evaluation value $y' \neq f(x)$ at the same point x . More formally, there exists an extractor that, given oracle access

to any prover that produces valid openings, can extract the underlying polynomial f .

- *Hiding*: If the scheme is hiding, the commitment σ reveals no information about f beyond what is revealed by evaluation openings.
- *Zero-Knowledge*: Ensures that evaluation proofs leak no additional information about the committed polynomial. There exists a simulator that can produce indistinguishable proofs given only the evaluation output $y = f(x)$.

C. Interactive proof & ZK proof systems

Interactive Proofs. An interactive proof is a protocol between two parties: a computationally unbounded *prover* P and a probabilistic polynomial-time *verifier* V . The verifier seeks to determine whether a given statement is true, based on the information provided by the prover through multiple rounds of interaction. In each round, V sends a message (query), and P replies, possibly adapting its answers based on the verifier's previous messages.

Formally, the prover attempts to convince the verifier that a computation $C(x) = y$ was performed correctly. We define this as follows:

Definition 1 (Interactive Proof). *Let $C : \mathcal{X} \rightarrow \mathcal{Y}$ be a deterministic function. A pair of interactive machines $\langle P, V \rangle$ is an interactive proof system for C with soundness error ϵ if the following two properties hold:*

- *Completeness. For all $x \in \mathcal{X}$ with $C(x) = y$, an honest prover convinces the verifier with probability 1:*

$$\Pr[\langle P, V \rangle(x) = \text{accept}] = 1.$$

- *ϵ -Soundness. For all $x \in \mathcal{X}$ and all (possibly malicious) provers P^* , if $y \neq C(x)$, then the verifier accepts with probability at most ϵ :*

$$\Pr[\langle P^*, V \rangle(x) = \text{accept}] \leq \epsilon.$$

An interactive proof system is said to have *succinct verifier time and proof size* if the communication complexity and verifier's running time are $\mathcal{O}(\text{polylog}(|C|, |x|))$.

Sumcheck Protocol. The sumcheck protocol [31], is a fundamental interactive proof protocol that enables a prover to convince a verifier of the claimed value of a sum of evaluations of a multivariate polynomial over the Boolean hypercube. It plays a central role in numerous interactive and ZKP systems, particularly those that require verifying computations over exponentially large domains.

Let $f : \mathbb{F}^\ell \rightarrow \mathbb{F}$ be a multivariate polynomial of individual degree at most d , and let the prover claim that

$$H = \sum_{\mathbf{b} \in \{0,1\}^\ell} f(\mathbf{b}).$$

Here, H denotes the prover's claimed value of the sum of a multivariate polynomial over the Boolean hypercube. The goal of the sumcheck protocol is for the prover to convince the verifier of the correctness of this claim via an interactive

protocol with ℓ rounds. At a high level, the protocol iteratively reduces the ℓ -dimensional sum into a sequence of one-dimensional checks. In each round $i \in [\ell]$, the prover sends a univariate polynomial $g_i \in \mathbb{F}[X_i]$ that encodes a partial sum of f , and the verifier checks a consistency condition between g_i and the previous round's claim. The verifier then samples a random challenge $r_i \in \mathbb{F}$ to fix one variable and proceed to the next round. In the final round, the verifier checks the correctness of the claimed evaluation of f at the random point $\mathbf{r} = (r_1, \dots, r_\ell)$. For more details of the sumcheck protocol see Fig. 5 in Appendix B.

Evaluation Reduction Sumcheck. Verifying k distinct evaluations $\{(x_i, y_i)\}_{i=1}^k$ of a multilinear polynomial f individually is computationally expensive. We instead employ the evaluation reduction technique [12], which aggregates these claims into a single consistent check. The verifier first samples random scalars $\alpha \in \mathbb{F}^k$ to compute a target value $T = \sum_{i=1}^k \alpha_i y_i$.

Recall from the above description of the sum-check protocol that the evaluation of any multilinear extension f at a point x is equivalent to its summation over the Boolean hypercube weighted by the identity function $\tilde{\beta}$. By substituting this definition into the target linear combination and exploiting the linearity of the summation, we can rearrange the terms as follows:

$$\sum_{i=1}^k \alpha_i f(x_i) = \sum_{z \in \{0,1\}^\ell} f(z) \cdot \underbrace{\left(\sum_{i=1}^k \alpha_i \cdot \tilde{\beta}(z, x_i) \right)}_{L(z)}. \quad (1)$$

Consequently, the validity of all k evaluations reduces to a *single* execution of the sumcheck protocol for the relation $\sum_{z \in \{0,1\}^\ell} f(z) \cdot L(z) = T$. This allows the verifier to outsource the computation of the combined basis polynomial $L(z)$ to the prover, reducing the verification overhead to that of a single point evaluation.

The GKR protocol. The GKR protocol [12], is an interactive proof protocol for verifying computations represented as layered arithmetic circuits over a finite field $\mathbb{F}$. Let C be such a circuit of depth d , where each layer $i \in \{0, \dots, d\}$ contains S_i gates, and each gate in layer i receives inputs from two gates in layer $i+1$. Without loss of generality, assume that S_i is a power of two, and define $s_i := \lceil \log S_i \rceil$.

We denote by $v_i : \{0,1\}^{s_i} \rightarrow \mathbb{F}$ the function that maps a binary gate label z to the output of the z -th gate in layer i . The wiring pattern of the circuit is described using predicate functions $\text{add}_i, \text{mult}_i : \{0,1\}^{s_i+2s_{i+1}} \rightarrow \{0,1\}$, where $\text{add}_i(z, x, y) = 1$ (respectively, $\text{mult}_i(z, x, y) = 1$) if gate z in layer i is an addition (resp., multiplication) gate that takes gates x and y in layer $i+1$ as input.

Given a fixed input to the circuit, the values v_i satisfy the following recurrence relation:

$$v_i(z) = \sum_{x,y \in \{0,1\}^{s_{i+1}}} [\text{add}_i(z, x, y) \cdot (v_{i+1}(x) + v_{i+1}(y)) + \text{mult}_i(z, x, y) \cdot v_{i+1}(x) \cdot v_{i+1}(y)]. \quad (2)$$

The GKR protocol allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ of the correct evaluation of C on a given input. Assuming $\mathcal{V}$ has oracle access to the input layer v_d and the claimed output v_0 , the protocol proceeds recursively from the output layer to the input layer. In each step, the verifier samples a random point $r_i \in \mathbb{F}^{s_i}$ and uses the sumcheck protocol (see Fig. 6 in Appendix B) to verify the correctness of Equation (2) at $z = r_i$. At the conclusion of the sumcheck, $\mathcal{V}$ queries the prover for evaluations of v_{i+1} at a small number of points, which are then combined via a random linear combination to reduce to a single claim at layer $i + 1$.

This process is repeated for d iterations until reaching the input layer, where the verifier can directly evaluate the final claims using its oracle access to v_d . The GKR protocol achieves logarithmic verifier time and communication in the size of the circuit and is a foundational tool in many modern zero-knowledge proof systems. To achieve zero-knowledge, the protocol can be combined with ZK sumcheck protocols [32], [33] and ZK polynomial commitment schemes. Furthermore, the protocol can be made non-interactive in the random oracle model via the Fiat-Shamir transform [34].

D. Recurrent Neural Networks

A recurrent neural network (RNN) [35] is a type of neural network designed to process sequential data by maintaining a hidden state that captures information from previous elements in the sequence. Unlike feedforward networks such as convolutional neural networks (CNNs), which process inputs independently, RNNs incorporate temporal dynamics by feeding the output from previous time steps back into the model as input. An RNN processes a sequence of inputs $x_1, x_2, \dots, x_T$ by maintaining a hidden state vector a_t at each time step t , which is updated based on the current input x_t and the previous hidden state a_{t-1} . The general update rule is:

$$a_t = F_1(W_{aa}a_{t-1} + W_{ax}x_t + b_a),$$

where W_{aa} and W_{ax} are weight matrices, b_a is a bias vector, and F_1 is a non-linear activation function such as tanh or ReLU. The hidden state h_t can be used to produce an output y_t as follows:

$$y_t = F_2(W_{ya}a_t + b_y),$$

where W_{ya} is an output weight matrix, b_y is an output bias, and F_2 is typically a softmax function for classification or a linear function for regression.

Sequential Computation. The computation in an RNN is inherently sequential. To compute h_t , one must first compute h_{t-1} , and so on, starting from an initial state h_0 . This sequential dependency poses challenges for parallelization but allows RNNs to capture long-range dependencies in the data. Figure

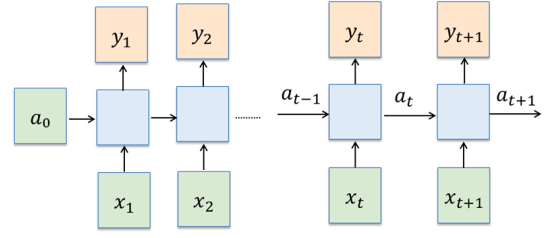


Fig. 1. Structure of a simple RNN with sequential input and hidden state propagation.

1 illustrates the structure of a simple RNN with sequential input and hidden state propagation.

Variants. Several extensions of RNNs have been proposed to address issues like vanishing and exploding gradients in long sequences. Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs) introduce gating mechanisms to better control information flow and preserve gradients. For more information, please check Appendix C. In this work, we present an inference framework that supports major RNN variants, including vanilla RNNs, GRUs, and LSTMs (see Appendix D). To demonstrate the framework's adaptability, we provide a specific and detailed inference protocol for the vanilla RNN, designed to facilitate efficient ZKP constructions.

III. ZERO-KNOWLEDGE PROOFS FOR RECURRENT NEURAL NETWORK

A. Handling Fixed-Point Arithmetic

To perform verifiable computation over real-valued RNN models, we must represent floating-point weights, inputs, and intermediate values within a finite field $\mathbb{F}_p$. We utilize the fixed-point encoding scheme in [11], where a real number $r \in \mathbb{R}$ is quantized into an integer $s \in \mathbb{F}_p$ using q -bit signed integer representation and f -bit fractional precision. Specifically, for $0 \leq r \leq 2^{q-f}$, we map r to $s = \lfloor r \cdot 2^f \rfloor \in \mathbb{F}_p$, and for $-2^{q-f} \leq r < 0$, we map r to $s = p - \lfloor r \cdot 2^f \rfloor$. This encoding ensures that basic arithmetic operations remain consistent over $\mathbb{F}_p$, while the fixed field size p is chosen sufficiently large to avoid overflow during RNN inference.

Arithmetic operations. Addition and subtraction are performed directly over the quantized values. For multiplication, suppose s_a, s_b are the fixed-point encodings of a and b , respectively. Their product $z = s_a \cdot s_b$ corresponds to $a \cdot b \cdot 2^{2f}$ in real arithmetic. To scale the result back to fixed-point precision, we divide z by 2^f . To enable verifiability, the prover provides the bit-decomposition $z_0, \dots, z_{q-1}$ of z , along with a sign bit z_q , where $z_q = 1$ denotes negativity. The verifier checks that each $z_i \in \{0, 1\}$ and that $z = (-1)^{z_q} \cdot \sum_{i=0}^{q-1} z_i \cdot 2^i$. The fixed-point product is then recovered as $(-1)^{z_q} \cdot \sum_{i=0}^{q-f-1} z_{i+f} \cdot 2^i$.

Comparisons. To evaluate comparisons (e.g., for conditional activation functions), the prover again provides a bit-decomposition of z including the sign bit z_q . The verifier uses this sign bit to determine whether $z \geq 0$, thereby supporting non-linear branching logic within the arithmetic circuit.

Division. Division operations are handled similarly. To compute a/b , given their quantizations $s_a, s_b \in \mathbb{F}_p$, the prover provides a candidate quotient z and remainder r , such that $s_a = z \cdot s_b + r$, with $0 \leq r < s_b$. The verifier checks this relation and returns $z \cdot 2^f$ as the scaled fixed-point quotient.

Scaling optimization. To minimize circuit complexity, we accumulate scaling factors within each computation block and defer scaling to the output of each layer in the RNN step. That is, intermediate products are maintained in scaled form within a layer, and a single rounding/truncation is applied at the layer output (e.g., after computing (z^t) and after computing (y^t)).

Sumcheck for Binary Operations. In order to support non-linear operations such as ReLU and tanh, which require comparison logic (e.g., sign extraction), we represent values using bit-decompositions. While these bit-level constraints—such as enforcing binary values and validating bit-to-integer consistency—can be embedded into generic arithmetic circuits, doing so significantly inflates circuit size and wiring predicates, especially in large or complex models. To alleviate this overhead, we follow the approach of [11] and express such binary constraints as *sumcheck instances* rather than embedding them directly into the circuit. Specifically, let $\mathbf{z} \in \mathbb{F}^n$ be a vector of prover-supplied values for which we must verify bit-decompositions. Let $\mathbf{Z}_{\text{bit}} \in \mathbb{F}^{n \times q}$ denote the claimed bit-decompositions and $\mathbf{z}_{\text{sign}} \in \mathbb{F}^n$ the associated sign bits. To verify that all entries in $\mathbf{Z}_{\text{bit}}$ and $\mathbf{z}_{\text{sign}}$ are binary, the verifier sends random challenges α, r_x, r_y , and the parties execute the sumcheck protocol on the following relation:

$$0 = \sum_{x \in \{0,1\}^{\log n}} \sum_{y \in \{0,1\}^{\log q}} \beta_x(r_x) \beta_y(r_y) (\alpha \mathbf{z}_{\text{sign}}(x) (1 - \mathbf{z}_{\text{sign}}(x)) + \mathbf{Z}_{\text{bit}}(x, y) (1 - \mathbf{Z}_{\text{bit}}(x, y))). \quad (3)$$

where β_x and β_y are multilinear basis polynomials. Then, to check that the bit-decomposition correctly reconstructs the integer value $\mathbf{z}$, the verifier sends another challenge r_z and the parties run a second sumcheck instance on:

$$\mathbf{z}(r_z) = \sum_{x \in \{0,1\}^{\log n}} \sum_{y \in \{0,1\}^{\log q}} \beta_x(r_x) \beta_y(r_y) \mathbf{Z}_{\text{bit}}(x, y) \cdot (1 - 2 \cdot \mathbf{z}_{\text{sign}}(x)) \cdot 2^y. \quad (4)$$

This construction ensures both binary validity and consistency between the value and its decomposition without expanding the underlying arithmetic circuit. Scaling operations can be handled similarly by adjusting the ranges of x and y in Equation 4 to target specific subsets of the bit-decomposed values and their scaling factors. This optimization is particularly valuable for supporting non-linear activation functions within a GKR-style framework.

Handling Exponentiation. Exponential functions arise in many non-linear layers, such as Softmax and tanh, and therefore must be supported by the proof system. However, directly evaluating exponentials such as e^x in an arithmetic circuit is expensive and inefficient over finite fields. Following prior work [36], [37], we approximate exponentiation using a

piecewise-linear function that is efficient to compute and prove inside zero-knowledge protocols. In practice, weights and data items are typically normalized to the interval $[-1, 1]$. In this setting, we use the following piecewise approximation of e^x , adapted from [11]:

$$e^x \approx \begin{cases} 0 & \text{if } x < -\frac{1}{2}, \\ \frac{1}{2} + x & \text{if } -\frac{1}{2} \leq x \leq \frac{1}{2}, \\ 1 & \text{if } x > \frac{1}{2}. \end{cases}$$

This approximation avoids costly exponentiation gates while maintaining sufficient accuracy for most inference applications.

B. Each Step Proof

At each time step t of the vanilla RNN, the hidden state a_t and output y_t are computed as:

$$a_t = F_1(W_{aa} \cdot a_{t-1} + W_{ax} \cdot x_t + b_a), \quad y_t = F_2(W_{ya} \cdot a_t + b_y)$$

where F_1 and F_2 are non-linear activation functions (e.g., ReLU or tanh). We aim to construct a succinct ZKP for each time step to ensure that the above computations are performed correctly and consistently with committed values. Our proof for a single time step, formalized in Fig. 2, adopts a modular strategy that decomposes the computation and leverages specialized sumcheck-based protocols for each component. This hybrid approach is designed to maximize prover efficiency by utilizing the most suitable cryptographic tool for each distinct task: a time-optimal sumcheck protocol for linear algebra, and a GKR-style proof for non-linear logic.

1) *Unified Proofs for Linear Operations:* A standard practice in modern ML systems is to reformulate computational steps as matrix-matrix or matrix-vector multiplications, motivated by the hardware efficiency of GPUs and other accelerators. We adopt this strategy to create a more efficient and simpler proof system. Consider the standard recurrent update equation $z_i = W_{ax}x_i + W_{aa}a_{i-1} + b_a$. Instead of proving three separate operations (two matrix-vector products and one vector addition), we express this as a single, larger matrix-vector product. We first construct an *augmented input vector* that concatenates the current input, the previous hidden state, and a constant “1” for the bias term:

$$\tilde{x}'_i = \begin{bmatrix} x_i \\ a_{i-1} \\ 1 \end{bmatrix} \quad (5)$$

Correspondingly, we define an *augmented weight matrix* that combines the respective weight matrices and the bias vector:

$$\tilde{W}_a = [W_{ax} \quad W_{aa} \quad b_a] \quad (6)$$

The entire linear transformation for the hidden state is then compactly written as $z_i = \tilde{W}_a \tilde{x}'_i$. This reformulation is applied to all linear steps in the RNN, including the output computation. By unifying the linear components this way, we can prove all of them using a single cryptographic primitive: Thaler’s time-optimal sumcheck protocol for matrix multiplication (SumMatMul) [16]. As detailed in Steps 2 and 4 of

Protocol A: Proof of RNN Step

Parameters. Let SumMatMul be the time-optimal sumcheck protocol for matrix multiplication [16]. All linear operations in the RNN step are reduced to a single matrix multiplication instance and proven by invoking SumMatMul. Let GKR be a generic GKR-style ZKP system which uses the optimized sumcheck protocol [17] as a subroutine. Let PCS = (KeyGen, Commit, Open, Verify) be the Orion polynomial commitment scheme [15].

Initialization: On inputs x_i and previous state a_{i-1} , $\mathcal{P}$ computes the RNN step: linear state $z_i = W_{ax}x_i + W_{aa}a_{i-1} + b_a$, state $a_i = F_1(z_i)$, linear output $s_i = W_{ya}a_i + b_y$, output $y_i = F_2(s_i)$, and auxiliary data AUX_i (bit-decompositions). $\mathcal{P}$ sends $\mathcal{V}$ commitments $\sigma_{a_i} \leftarrow \text{PCS.Commit}(a_i, r_{a_i})$ and $\sigma_{AUX_i} \leftarrow \text{PCS.Commit}(AUX_i, r_{AUX_i})$. (Commitments to weights σ_W and previous state $\sigma_{a_{i-1}}$ are known).

Phase 1: Proof of Output Layer.

- 1) $\mathcal{V}$ samples r_y and requests evaluation $\tilde{y}_i(r_y)$.
- 2) Having $\tilde{y}_i(r_y)$, parties run GKR on $y_i = F_2(s_i)$. The circuit uses bit-decompositions from AUX_i . This ends with evaluations $\tilde{s}_i(r_{s_i})$ and $AUX_i(r_{AUX,1})$.
- 3) Having $\tilde{s}_i(r_{s_i})$, parties run SumMatMul on $s_i = W_{ya}a_i + b_y$. At the end, parties receive evaluations $\tilde{a}_i(r_{a,\text{out}})$, $\tilde{W}_{ya}(r_{W_{ya}})$, and $\tilde{b}_y(r_{b_y})$.

Phase 2: Proof of State Layer.

- 1) Parties run GKR on $a_i = F_1(z_i)$ using $\tilde{a}_i(r_{a,\text{out}})$ as the claim. This ends with evaluations $\tilde{z}_i(r_{z_i})$ and $\widetilde{AUX}_i(r_{AUX,2})$.
- 2) Having $\tilde{z}_i(r_{z_i})$, parties run SumMatMul on $z_i = W_{ax}x_i + W_{aa}a_{i-1} + b_a$. At the end, parties receive evaluations $\tilde{x}_i(r_{x_i})$, $\tilde{a}_{i-1}(r_{a_{i-1}})$, $\tilde{W}_{ax}(r_{W_{ax}})$, $\tilde{W}_{aa}(r_{W_{aa}})$, and $\tilde{b}_a(r_{b_a})$.

Phase 3: Consistency Checks.

- 1) Parties run sumchecks (analogous to Eq 3, 4) to prove bit-decomposition correctness for AUX_i . This ends with evaluations combined into a single $\widetilde{AUX}_i(r_{AUX,3})$.

Phase 4: Evaluation Reduction.

- 1) Parties combine received evaluations of $\widetilde{AUX}_i$ from Phases 1, 2, and 3 into a single evaluation $\widetilde{AUX}_i(r_{\text{in},AUX})$.
- 2) The final set of evaluations $\mathcal{E}^{\text{raw}} = \{\tilde{a}_{i-1}, \tilde{x}_i, \tilde{a}_i, \tilde{W}_{\dots}, \widetilde{AUX}_i\}$ and raw wiring predicates $\mathcal{W}^{\text{raw}}$ (and $\pi_{i,G}$ if non-interactive) are returned. These are verified by invoking PCS.Open or passed to the aggregation scheme.

Fig. 2. Proof of RNN Step

Protocol (cf. Fig. 2), this protocol is invoked to prove the correctness of these unified matrix products. This approach is faster than arithmetizing the operations in a generic circuit and allows us to prove the most computationally intensive part of the RNN step efficiently.

2) *Proofs for Non-Linear Activations:* While the unified matrix multiplication handles the linear layers, the non-linear activation functions, e.g., $a_i = F_1(z_i)$ and $y_i = F_2(s_i)$, which typically represent functions like ReLU or tanh, require a different approach. For these, we rely on a generic GKR-style proof system operating on a layered arithmetic circuit. Following the methodology of prior work [11], [37]–[39], the GKR circuit itself manages these non-arithmetic operations using the bit-decomposition and piecewise linear approximations detailed in Section III-A. The prover provides the binary representation of the inputs to the non-linear functions (e.g., the bits of z_i) as part of a committed auxiliary witness, AUX_i . The GKR proof, which we accelerate with the optimized sumcheck from Dao and Thaler [17], then operates on these bits to verify the function’s logic. For example, a ReLU function ($\max(z, 0)$) is verified by checking the sign bit provided in the decomposition. A critical component is proving that this auxiliary data is correct. To ensure the prover cannot cheat by providing invalid bits, we include dedicated sumcheck proofs. These consistency checks are expressed *directly as sumcheck instances* rather than being arithmetized,

which avoids a significant blow-up in the circuit size and formally attests to the consistency between the original values and their committed bit-decompositions.

C. Recursive Proof Composition

Verifying an inference sequence of length T by simply concatenating T instances of the step-proof described in Section III-B would require the verifier to verify each time step individually. This linear scaling in verification time and proof size is neither practical nor efficient for long sequences. Instead, zkRNN employs a recursive composition strategy based on IVC [30].

Incrementally Verifiable Inference. We construct our recursive proof system by defining an augmented transition function, denoted as F_A . The augmented function F_A accepts cryptographic proofs from its children, denoted as $\{\pi_{\text{child}}\}_{j=1}^k$. The execution of F_A at a *node* checks that the child proof packages are valid (when present). In particular, at a leaf node, simply the RNN step for that time index will be computed, and a leaf proof will be produced; at an internal node, first F_A verifies its k children if verification passes, they will be aggregated to the parent proof. This structure ensures that a single proof at the root inductively implies the correctness of the entire inference history covered by the leaves beneath it [30].

A critical challenge in realizing F_A is the cost of verifying child ZK arguments inside the arithmetic circuit. Each π_{child} consists of transcripts from the GKR and matrix multiplication sumcheck protocols used to validate the linear and non-linear operations of the RNN layer. To verify π_{child} , the augmented circuit must perform *In-Circuit Sumcheck Verification*. This involves simulating the honest verifier’s protocol: deriving random challenges via the Fiat–Shamir transformation and checking the arithmetic identities of the sumcheck protocol. However, the standard verifier also requires checking polynomial commitment openings, which relies on heavy cryptographic hashing. To maintain efficiency, our in-circuit verifier performs only the arithmetic checks and challenge generation, deferring the expensive polynomial opening checks via the aggregation techniques described next.

Evaluation reduction and aggregated openings. zkRNN employs a unified mechanism for compressing many commitment-evaluation claims into a single final opening. The approach follows the polynomial-commitment aggregation method introduced in [11], whose full structure is recalled in Fig. 7 in Appendix B. Here we present a concise description of the underlying ideas before explaining how Protocol B, presented in Fig. 4, incorporates them into the zkRNN proof system.

The aggregation method begins by reducing multiple evaluation checks to one. Suppose a collection of polynomials $f_1, \dots, f_k$ is committed, and for each we must verify an evaluation $y_i = f_i(x_i)$. The verifier and prover construct a univariate interpolation map L satisfying $L(i) = x_i$ and require the prover to supply g_i , a univariate polynomial meant to represent $f_i \circ L$. After verifying that $g_i(i) = y_i$ for all i , the verifier samples random coefficients and a random point, forming a single pair (x^*, y^*) defined by $x^* = L(r)$ and $y^* = \sum_i \alpha_i g_i(r)$. The prover forms an aggregate polynomial $f^* = \sum_i \alpha_i f_i$, whose correctness at the point x^* is sufficient to ensure correctness of all original evaluations except with negligible probability. To avoid revealing structural information about each f_i , random masking polynomials are added to the compositions prior to aggregation, ensuring the reduction step remains zero-knowledge.

The second phase aggregates the commitments themselves. The prover computes a commitment σ^* to f^* and must now prove that this commitment is consistent with the intended linear combination of the original commitments. As shown in Fig. 7 (cf. Appendix B), this is achieved by opening a small, verifier-chosen set of positions in the encoded representations underlying the commitments. Because the encoding used in the commitment scheme is linear, the corresponding positions in the encoding of f^* must match the same linear combination of the sampled positions in the encodings of the input polynomials (including the masks). Coding-theoretic properties guarantee that any deviation from the prescribed relation affects a substantial fraction of positions, and therefore a logarithmic number of random checks suffices for soundness.

Protocol B (see Fig. 4) applies this aggregation procedure incrementally across the recurrent computation. At each time

step, the RNN transition proof yields several committed polynomials and associated evaluation claims. These are merged with the previously maintained aggregate triple by invoking the same evaluation-reduction and commitment-aggregation steps described above. The outcome is a new tuple $(\sigma_t^*, x_t^*, y_t^*)$ that summarizes all commitment–evaluation relations up to that point, allowing the verifier to discard the individual claims. After the final step, the verifier performs a single opening of the final aggregate commitment at the associated point (x_T^*, y_T^*) . By the security of the aggregation method [11], this opening certifies the correctness of every commitment–evaluation pair encountered throughout the entire execution, yielding a succinct and sound verification procedure based on aggregated openings.

Public Input Aggregation. To compress multiple claims about the public inputs $\{(x_i, y_i)\}_{i=1}^k$ into a single verifiable claim, we employ the polynomial aggregation technique described in [11]. Let f_i denote the Multilinear Extension (MLE) corresponding to the i -th public input x_i . The prover and verifier interpolate a low-degree map $L : \mathbb{F} \rightarrow \mathbb{F}^\ell$ such that $L(i) = x_i$ for all $i \in \{1, \dots, k\}$. A combined polynomial $g(t)$ is then defined as a random linear combination of the composition of the original functions with L :

$$g(t) := \sum_{i=1}^k \alpha_i \cdot (f_i \circ L)(t) = \sum_{i=1}^k \alpha_i \cdot f_i(L(t)), \quad (7)$$

where $\{\alpha_i\}$ are random coefficients sampled by the verifier. The validity of the k individual claims reduces to checking a single evaluation at a random challenge point $r \in \mathbb{F}$. Specifically, the verifier checks the aggregated claim $g(r) = y^*$, where $y^* = \sum \alpha_i y_i$. This reduces the verification of k distinct input vectors to verifying a single evaluation of the aggregated polynomial g at r .

Wiring Aggregation. To avoid inflating the augmented circuit F_A with the arithmetic logic required to evaluate wiring predicates (e.g., $\widehat{\text{add}_i}, \widehat{\text{mult}_i}$), we delegate these evaluations to the prover. Instead of computing them natively, the circuit treats the results as claims and employs the *Evaluation Reduction Sumcheck* (II-C) to compress them into a single aggregate claim wp_{agg} . This aggregate is propagated via the accumulator and verified solely at the recursion root, effectively decoupling the verification complexity from the circuit topology [11], [40].

Tree-based organization. The multivariate commitment aggregation scheme guarantees knowledge soundness primarily for a constant number of recursive folds [11]; deep sequential recursion can degrade these guarantees and potentially expose the system to attacks [41]. To mitigate this, we adopt a tree-based recursion strategy [42] described in Fig. 3, where leaves represent per-iteration step proofs. Each internal node verifies its k child proofs in-circuit and reapplies the reduction and aggregation pipeline to emit a single parent proof. This topology reduces the recursion depth to $O(\log_k T)$, requiring the external verifier to check only the final proof at the root.

a) Proof packaging and optional compression. After constructing the k -ary recursion tree and producing the

final root proof package π_{root} , we optionally apply a black-box, lossless compression step before transmission, outputting $\pi_{\text{zip}} \leftarrow \text{Compress}(\pi_{\text{root}})$. The verifier first decompresses, $\pi_{\text{root}} \leftarrow \text{Decompress}(\pi_{\text{zip}})$, and then runs the standard root verification on π_{root} ; since decompression exactly reconstructs the original proof, this post-processing preserves completeness, knowledge soundness, and zero-knowledge. We include compression solely to reduce bandwidth: zip reports that serialized proofs for hash-based SNARGs exhibit significant redundancy and that applying simple lossless compressors typically reduces proof size by approximately 5–40% (with best cases near $\sim 60\%$ of the original), while adding only millisecond-scale encoding/decoding latency [43]. Applying the same post-processing to zkRNN therefore yields strictly smaller proofs on the wire without changing cryptographic assumptions or touching the recursive construction.

D. Putting Everything Together

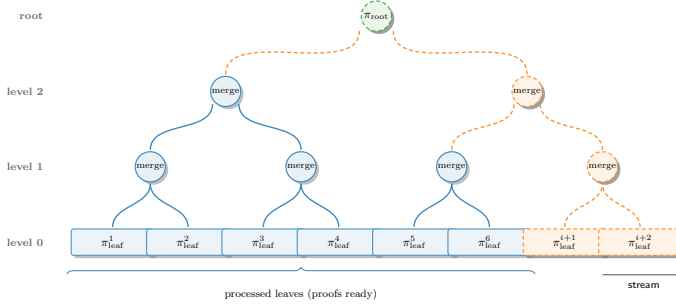


Fig. 3. Streaming proof generation using a k -ary tree (shown for $k = 2$). Leaf proofs (π_{leaf}) are incrementally merged as the stream continues (dashed lines), culminating in a single root proof (π_{root}).

To prevent verification cost from scaling linearly with the number of steps, zkRNN defers commitment openings by folding these per-step claims into accumulators. Concretely, after Protocol A (cf. Fig. 2), the prover runs the aggregation prover AGG.P on $\mathcal{E}^{\text{raw}}$ to obtain a single data accumulator agg_i together with a short aggregation proof and an auxiliary witness that can be carried into the next recursion layer. Intuitively, agg_i summarizes all commitment–evaluation relations generated at step i in a form that can be merged with other summaries, enabling the protocol to postpone the expensive polynomial-opening verification to a single check at the end. In the same spirit, wiring predicate verification is also deferred: the wiring claims $\mathcal{W}^{\text{raw}}$ are compressed into a single wiring accumulator wp_i using evaluation reduction, so that wiring consistency can be checked once at the root. The resulting leaf proof package for step i therefore contains (i) the sumcheck/GKR transcript component for the step, (ii) the aggregated data accumulator and its proof/witness, and (iii) the aggregated wiring claim.

Succinctness over long streams is obtained by recursively composing these leaf packages in a k -ary tree as specified in Protocol B (cf. Fig. 4). The system maintains a small buffer of pending proof packages at each recursion level; each new

leaf is placed into the level-0 buffer, and whenever any level accumulates k children it is immediately merged into a parent package, continuing eagerly as the stream progresses. A parent package is defined by an augmented function F_A that verifies and combines its k children. Inside F_A , the required Fiat–Shamir challenges and hash computations are recomputed, and each child’s arithmetic transcript is verified in-circuit (i.e., the parent proves that it performed the same checks an external verifier would perform for the child’s sumcheck/GKR components). Crucially, polynomial openings are not verified in-circuit; instead, successful in-circuit verification produces the new evaluation claims implied by the child computation, and the parent additionally verifies that the child’s aggregation proof correctly folded its input accumulators and these new claims into the child’s output accumulator. The parent then produces its own transcript proof $\pi_{\text{parent},G}$ attesting to correct execution of F_A , merges the children’s data accumulators into $\text{agg}_{\text{parent}}$ via AGG.P, and folds the children wiring accumulators into $\text{wp}_{\text{parent}}$. By using a tree of arity $k \geq 2$, the recursion depth is $O(\log_k T)$.

Because the inputs x_i form a potentially unbounded public stream, zkRNN also aggregates public inputs alongside commitment-related claims so that the proof does not grow with the stream length. Using interpolation-based public input aggregation, the protocol compresses groups of inputs into a single aggregate claim that is merged up the recursion tree in the same way as the data accumulators. At verification time, the verifier checks this final public-input aggregate against the aggregate it computes locally from the stream prefix, ensuring that the root proof corresponds to the same sequence of public inputs that produced the online outputs.

When the user requests a proof for the processed prefix, the system merges all buffered nodes into a single root package (padding if necessary) and optionally compresses it for transmission. The verifier decompresses, recomputes all Fiat–Shamir challenges and hashes, and verifies the root transcript component, which certifies that the root executed F_A correctly and thereby that all child transcript verifications and accumulator-validations were carried out correctly throughout the tree. The verifier then checks the final wiring accumulator against the known wiring predicates, verifies the aggregation proof showing that agg_{root} correctly summarizes all evaluation claims induced across the entire computation, and verifies the aggregated public-input claim against its locally maintained stream aggregate. Finally, the verifier performs a single polynomial-opening verification using PCS.Verify for agg_{root} . This last check binds all deferred commitment–evaluation relations to their underlying commitments, and together with the recursive transcript verification implies that every reported output y_i is consistent with correct execution of the RNN recurrence under the committed model and initial state, while intermediate states and auxiliary witnesses remain hidden.

Theorem III.1. *The Proof of RNN Step protocol (Protocol A described in Fig. 2) satisfies completeness, knowledge soundness, and zero-knowledge.*

Protocol B: Streaming and Recursive ZK Proof of RNN Inference

Parameters. Let SumMatMul be the time-optimal sumcheck protocol for matrix multiplication [16]. Let GKR be a generic GKR-style zero-knowledge proof system which uses the optimized sumcheck protocol [17] as a subroutine. Let PCS = (KeyGen, Commit, Open, Verify) be the Orion polynomial commitment scheme [44]. Let AGG = (Prove, Verify) be the non-interactive aggregation scheme [11]. We specify that AGG supports Public Input Aggregation via Interpolation to compress public input streams $\{x_i\}_{i=1}^T$ alongside polynomial commitments. Let ProofOfRNNStep be the step-proof protocol described in Fig. 2. Let (Compress, Decompress) be a lossless compressor [43]. Let $k \geq 2$ be the recursion arity (the number of children per internal node). Let π_0 be a trivially satisfying proof package.

Model and state. Model commitments $\sigma_W = \{\sigma_{W_{ax}}, \sigma_{W_{aa}}, \sigma_{W_{ya}}, \sigma_{b_a}, \sigma_{b_y}\}$. Initial state a_0, σ_{a_0} .

Goal. The system processes a stream of inputs $\{x_i\}_{i=1}^T$, providing immediate outputs $\{y_i\}_{i=1}^T$. At any point, the user can request a single, succinct root proof that validates the entire sequence of computations up to that point.

Streaming Orchestration The system maintains the current state (a, σ_a) and a buffer of pending proofs for each recursion level.

OnInput(x_i): For each input, the system generates a leaf proof π_{leaf} and output y_i via **ProveLeaf**, then immediately returns y_i . The new state (a_i, σ_{a_i}) is stored. The new π_{leaf} is buffered, triggering eager recursive merging via **ProveInternalNode** whenever any level accumulates k proofs.

FinalizeProof(): All buffered proofs are recursively merged up to the root via **ProveInternalNode** (padding if necessary), computing the final, single proof π_{root} . The compressed proof $\pi_{\text{zip}} \leftarrow \text{Compress}(\pi_{\text{root}})$ is computed. Return π_{zip} .

ProveLeaf($i, x_i, a_{i-1}, \sigma_W, \sigma_{a_{i-1}}, pp$): Generates the leaf proof for a single RNN step.

- 1) **Compute Witness:** Compute the RNN step $(z_i, a_i, s_i, y_i) \leftarrow \text{RNN}(x_i, a_{i-1}, W)$ and the auxiliary data AUX_i .
- 2) **Commit State:** Compute the new state commitments $\sigma_{a_i} \leftarrow \text{PCS.Commit}(a_i, r_{a_i}, pp)$ and $\sigma_{\text{AUX}_i} \leftarrow \text{PCS.Commit}(\text{AUX}_i, r_{\text{AUX}_i}, pp)$.
- 3) **Generate Step Proof:** Call Protocol A to get the raw proof and claims:

$$(\pi_{i,G}, \mathcal{E}^{\text{raw}}, \mathcal{W}^{\text{raw}}) \leftarrow \text{ProofOfRNNStep.Prove}(x_i, y_i, \sigma_W, \sigma_{a_{i-1}}, \sigma_{a_i}, \sigma_{\text{AUX}_i}).$$

- 4) **Generate Leaf Aggregates:** If needed, Run AGG.P on the evaluation instances $\mathcal{E}^{\text{raw}}$ per polynomial family, producing the tuple $(\text{agg}_i, \pi_{i,A}, \pi_{i,E})$ of per-family. Concurrently, apply evaluation reduction to $\mathcal{W}^{\text{raw}}$ to compute the wiring aggregate wp_i .
- 5) **Assemble and Return:** Define the proof package $\pi_{\text{leaf}} := (\pi_{i,G}, \pi_{i,A}, \pi_{i,E}, \emptyset, \text{agg}_i, \emptyset, \text{wp}_i)$ and Return $(a_i, \sigma_{a_{i-1}}, y_i, \pi_{\text{leaf}})$.

ProveInternalNode($\{\pi_{\text{child}_j}\}_{j=1}^k, pp$): This function generates a parent proof by verifying and aggregating k child proofs.

- 1) **Define the Augmented Function F_A (Verification).**
 - a) **Evaluate Hashes:** Evaluate Fiat-Shamir challenges and Merkle hashes required for verifying the proof components (e.g., $\pi_{j,G}, \pi_{j,A}$) within each child proof $\{\pi_{\text{child}_j}\}_{j=1}^k$.
 - b) **In-Circuit Verification.**
 - i) Parse the k child proofs $\{\pi_{\text{child}_j}\}_{j=1}^k$. Each π_j is parsed as $(\pi_{j,G}, \pi_{j,A}, \pi_{j,E}, \{\text{agg}_{j,\text{in},m}\}_{m=1}^k, \text{agg}_j, \{\text{wp}_{j,\text{in},m}\}_{m=1}^k, \text{wp}_j)$, where $\{\text{agg}_{j,\text{in},m}\}_{m=1}^k$ are the input aggregates to child j , and agg_j is its output aggregate (same for $\{\text{wp}_{j,\text{in},m}\}_{m=1}^k$ and wp_j).
 - ii) For each child $j \in [1, k]$:
 - A) **Verify Sumchecks & Wiring Predicate:** Run the verifier for $\pi_{j,G}$ (e.g., GKR.V, SumMatMul.V). This verification takes the input aggregates $\{\text{agg}_{j,\text{in},m}\}_{m=1}^k$ and $\{\text{wp}_{j,\text{in},m}\}_{m=1}^k$ along with other public data. Successful verification yields the set of data evaluation instances $\mathcal{E}_j^{\text{new}}$ generated within the child's step and the updated predicate aggregation wp_j .
 - B) **Verify Data Aggregation:** Run AGG.V on $\pi_{j,A}$ to confirm agg_j correctly aggregates its inputs, $\{\text{agg}_{j,\text{in},m}\}_{m=1}^k$ and $\mathcal{E}_j^{\text{new}}$.
 - iii) If all sumchecks (a) and aggregations (b) pass, the F_A computation is valid.
 - 2) **Generate and Assemble Parent Proof.** The prover now generates the components for the parent proof package.
 - a) set $u_{\text{parent}} = \sigma_{a_0} \parallel \sigma_{a_{\text{child}_1}} \parallel \dots \parallel \sigma_{a_{\text{child}_k}} \parallel \pi_{\text{child}_1} \parallel \dots \parallel \pi_{\text{child}_k}$ and commit to it $\sigma_{u_{\text{parent}}} \leftarrow \text{PCS.Commit}(u_{\text{parent}}, r_{u_{\text{parent}}}, pp)$.
 - b) **Prove Execution & Aggregate Wiring:** Generate a GKR/sumcheck proof $\pi_{\text{parent},G}$. This single proof attests to the correct execution of the entire F_A (from Step 1) and combine received wiring predicate evaluations into $\text{wp}_{\text{parent}}$ by applying evaluation reduction sumcheck. Similarly, combine all evaluations of u_{parent} . This proof includes the necessary sumchecks for all hash evaluations performed within F_A .
 - c) **Aggregate Data:** Generate the parent data aggregate $(\text{agg}_{\text{parent}}, \pi_{\text{parent},A})$ by running the aggregation prover, AGG.P on $\{\text{agg}_j, \pi_{j,E}\}_{j=1}^k$ and remaining commitments and their evaluations (such as u_{parent} 's). Also aggregate the public inputs $\{\hat{x}_i\}_{i=1}^k$ using III-C.
 - d) **Prepare Final Opening:** Generate the final proof component, $\pi_{\text{parent},E}$. If this is the root node, this component is the evaluation opening proof from PCS.Open($\text{agg}_{\text{parent}}$). Otherwise, it is the secret witness for $\text{agg}_{\text{parent}}$ required by the next level of recursion.
 - 3) **Return Parent Proof Package.** Assemble and return the complete proof $\pi_{\text{parent}} := (\pi_{\text{parent},G}, \pi_{\text{parent},A}, \pi_{\text{parent},E}, \{\text{agg}_j\}_{j=1}^k, \text{agg}_{\text{parent}}, \{\text{wp}_j\}_{j=1}^k, \text{wp}_{\text{parent}})$.

Verify($i, \sigma_W, \sigma_{a_0}, \sigma_{a_i}, \pi_{\text{zip}}, pp$):

- a) Decompress $\pi_{\text{zip}} \rightarrow \pi_{\text{root}}$. Parse π_{root} as $(\pi_G, \pi_A, \pi_E, \{\text{agg}_{\text{in},m}\}_{m=1}^k, \text{agg}_{\text{root}}, \{\text{wp}_{\text{in},m}\}_{m=1}^k, \text{wp}_{\text{root}})$.
- b) Evaluate all Fiat-Shamir and Merkle hashes for π_G and π_A .
- c) Run GKR.V/SumMatMul.V on π_G (using public inputs $i, \sigma_W, \sigma_{a_0}, \sigma_{a_i}, x_i$ and aggregates $\{\text{agg}_{\text{in},j}\}_{m=1}^k, \{\text{wp}_{\text{in},m}\}_{m=1}^k$) to verify the root node's computation. This check yields the new data claims $\mathcal{E}^{\text{root}}$ and the final wiring aggregate wp_{root} .
- d) Directly verify wp_{root} against the known wiring predicates.
- e) Run AGG.V on π_A to verify that agg_{root} correctly aggregates the child inputs $\{\text{agg}_{\text{in},j}\}_{m=1}^k$ and the new claims $\mathcal{E}^{\text{root}}$.
- f) Directly verify x_{root} against the known aggregate of $\{x_i\}_{i=1}^t$ which has been computed locally.
- g) If this is the final iteration Run PCS.Verify($\text{agg}_{\text{root}}, \pi_E$). Otherwise Check that π_E is a valid opening witness for agg_{root} .

Fig. 4. Streaming and Recursive ZK Proof of RNN Inference

Theorem III.2. *Protocol B, presented in Fig. 4 is a Zero-knowledge of RNN and satisfies completeness, knowledge soundness, and zero-knowledge.*

Due to space limitations, we present the proofs in Appendix E.

E. Complexity Analysis

We analyze the asymptotic complexity of zkRNN in terms of the sequence length T , hidden state size h , input dimension n , and quantization precision q . For the prover, each step encompasses a unified matrix-vector product with an $h \times (h+n)$ weight matrix together with the non-linear activation checks. The cost of the linear part is proportional to the number of parameters, $O(h^2 + hn)$, while the non-linear verification scales as $O(hq)$. Hence the total prover time over a sequence of length T is $O(T \cdot (h^2 + hn + hq))$, which is linear in the length of the RNN execution trace.

The verifier's complexity consists of processing the public input stream and checking the final recursive proof. Because $(x_1, \dots, x_T)$ is public, the verifier must read all inputs, incurring a cost of $O(Tn)$. The remaining work—verifying the internal computation of the RNN—is succinct in the size of the underlying circuit: the verifier checks a constant number of aggregated polynomial commitment openings whose cost is polylogarithmic in the number of circuit wires. Concretely, this yields $O(Tn + \log^2(h^2 + hn) + \log^2(hq))$. Thus, apart from the unavoidable linear pass over the input sequence, verification is succinct in the circuit size. The proof size is governed by the same aggregated openings and is likewise independent of the sequence length T . The final proof π_{root} contains a constant number of sumcheck transcripts and commitment openings, whose size is dominated by Merkle [45] authentication paths for the model parameters and auxiliary witnesses. Asymptotically, we obtain a proof size of $O(\log^2(h^2 + hn) + \log^2(hq))$, i.e., communication that is succinct in the circuit size and does not grow with T .

IV. EVALUATION

A. System Implementation

1) *Software:* We implemented the scheme using a streaming recursive proof architecture built on top of KAIZEN's cryptographic primitives [11]. The implementation is available at [46]. The system manages the recursive proof generation via a `StreamOrchestrator` that utilises multi-level buffers to process timesteps incrementally.

For proof generation, Orion [15] is utilised for polynomial commitments and generic GKR circuits to prove the correctness of non-linear activations (`tanh`, `softmax`). All computations are bridged to field arithmetic using fixed-point quantisation with Q fractional bits to ensure compatibility between the prover's tensor computations and the verifier's checks. The implementation optimises for memory by performing a single forward pass per timestep, reusing results for both execution and proof generation.

2) *Models:* We evaluate the system on three distinct RNN configurations of varying complexity to assess performance across different scales:

- 1) **Small:** A lightweight configuration with sequence length $T = 8$ and dimensions $n = m = k = 32$, resulting in approximately 0.01M parameters.
- 2) **Medium:** A standard configuration with $T = 32$ and dimensions $n = m = k = 128$, totaling 0.56M parameters.
- 3) **Large:** A complex configuration with $T = 64$ and dimensions $n = m = k = 128$, totaling 1.08M parameters.

We also vary the Arity (k') to evaluate the impact of the aggregation tree structure on prover efficiency.

3) *Hardware:* We run all experiments on a machine equipped with a 12th Generation Intel(R) Core i7-12650H processor (2.30 GHz) and 64GB of RAM. Our current implementation is not parallelised and utilises only a single CPU core. We report the average running time across multiple executions (ranging from 3 to 20 runs depending on the configuration) to ensure statistical reliability.

B. Performance

We evaluate zkRNN on several dimensions: (1) prover and verifier efficiency across model sizes, (2) scalability with respect to sequence length and arity, and (3) memory and proof size efficiency.

1) *Performance Analysis:* We first compare the performance of zkRNN across the Small, Medium, and Large configurations. As shown in Table I, Prover generation time varies significantly based on configuration size and aggregation strategy. This can be shown in Table I, whereby for each configuration setting, the prover and verifier times are averaged over multiple runs, with variability reported as the standard deviation. For small configurations ($T = 8, n = 32$), proof generation is negligible, requiring only 0.27 to 0.32 seconds. In medium configurations ($T = 32, n = 128$), execution time ranges from 45.8 to 217.3 seconds, showing a strong dependency on the chosen arity. Notably, for large configurations ($T = 64, n = 128$), the prover time stabilises at approximately 87.5 seconds for both arity 4 and arity 8 (averaged over 6-9 runs), suggesting that higher arities may offer better amortisation of overheads as sequence length increases.

A critical advantage of our approach is the extremely low verification overhead. Across all tested configurations, verifier time remains consistently fast, ranging between 0.997 and 1.545 milliseconds. This metric is effectively independent of both the sequence length (T) and RNN dimensions, empirically validating the logarithmic verification complexity inherent to the GKR protocol.

The generated proofs are highly compact, measuring between 17.66 KB and 21.97 KB across all experiments. The proof size remains nearly constant with respect to the sequence length T , a key property for blockchain-constrained environments. We observe a minor trade-off between aggregation arity and proof size, with an 11–24% increase in size when scaling from arity 2 to arity 8.

TABLE I
PERFORMANCE RESULTS

Configuration	T	n=m=k	Arity	Parameters	Prover (s)	Verifier (ms)	Proof Size (KB)	Peak Memory (MB)
Small-1	8	32	2	0.01M	0.315 ± 0.015	1.058 ± 0.084	17.66	9.22
Small-2	8	32	4	0.01M	0.268 ± 0.007	1.191 ± 0.066	19.69	10.24
Small-3	8	32	8	0.01M	0.276 ± 0.038	1.138 ± 0.087	18.16	12.29
Medium-1	32	128	2	0.56M	49.521 ± 9.591	0.981 ± 0.042	17.66	58.37
Medium-2	32	128	4	0.56M	45.828 ± 7.359	1.153 ± 0.048	19.69	56.32
Medium-3	32	128	8	0.56M	217.327 ± 57.892	1.512 ± 0.101	21.97	56.32
Large-1	64	128	2	1.08M	108.406 ± 6.380	1.089 ± 0.160	17.66	65.54
Large-2	64	128	4	1.08M	87.504 ± 10.470	1.015 ± 0.074	17.78	62.46
Large-3	64	128	8	1.08M	87.539 ± 8.631	1.003 ± 0.065	18.16	61.44

Memory usage is primarily determined by the RNN dimensions rather than the sequence length, scaling from 9 MB for small models ($n = 32$) to 60 MB for larger models ($n = 128$). Most importantly, doubling the sequence length from $T = 32$ to $T = 64$ resulted in only a 5% increase in peak memory usage. This minimal growth confirms the effectiveness of the streaming architecture, which processes the sequence incrementally without requiring the entire computation trace to be held in memory.

2) *Scalability Analysis*: We analyse the scalability of our system regarding sequence length, arity, and memory usage.

Theoretical predictions suggest prover time scales linearly with T . Our results support this for small sequences ($T = 8$). However, for the Large configuration ($T = 64$), the per-timestep cost drops significantly (1.37s vs 6.79s for Medium Arity 8). This suggests sub-linear scaling capabilities due to better amortisation of fixed costs (aggregation overhead, proof finalisation) in longer runs.

We also observe a significant trade-off regarding Arity. Higher arity reduces the aggregation tree depth ($O(\log_k T)$) but increases the per-aggregation cost. For $T = 32$, increasing arity from 2 to 8 resulted in a 4.4x slowdown (49.5s vs. 217.3s), indicating the quadratic cost of aggregation dominated. For $T = 64$, arity 4 and 8 performed nearly identically ($\approx 87.5s$). This suggests that for longer sequences, the aggregation tree structure becomes more balanced, neutralising the penalty of higher arity.

A critical property of the streaming architecture is memory efficiency. Memory usage scales with RNN dimensions ($O(n^2)$) rather than sequence length. Increasing T from 32 to 64 resulted in only a 5% memory increase (57 MB vs 60 MB), confirming the effectiveness of the streaming buffer design. However, the proof sizes remain compact (17-22 KB) and effectively constant regardless of sequence length through recursive aggregation.

3) *Limitations*: Our evaluation validates the new model for verifiable RNNs where every timestep is independently proven and aggregated via IVC. This architecture ensures that memory usage remains independent of sequence length, enabling the processing of long time-series data which is fundamentally not achievable by monolithic proofs. However, the approach introduces notable design considerations. First,

while verification is constant-time, the prover overhead for high-arity aggregations is significant, suggesting that lower arity (2–4) is optimal for current implementations. Second, our current evaluation relies on fixed 32-bit quantisation; exploring different precision levels is an important direction for future work. Finally, while we demonstrate scalability up to $T = 64$, further extensive benchmarks on larger datasets are required to fully characterise the sub-linear scaling behaviour observed in larger models.

REFERENCES

- [1] S. K. Mudgal, R. Agarwal, J. Chaturvedi, R. Gaur, and N. Ranjan, “Real-world application, challenges and implication of artificial intelligence in healthcare: an essay,” *Pan African Medical Journal*, vol. 43, no. 1, 2022.
- [2] I. Mathauer and M. Oranje, “Machine learning in health financing: benefits, risks and regulatory needs,” *Bulletin of the World Health Organization*, vol. 102, no. 3, p. 216, 2023.
- [3] Z. Ghodsi, T. Gu, and S. Garg, “Safetynets: Verifiable execution of deep neural networks on an untrusted cloud,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] L. Zhao, Q. Wang, C. Wang, Q. Li, C. Shen, and B. Feng, “Veriml: Enabling integrity assurances and fair payments for machine learning as a service,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 10, pp. 2524–2540, 2021.
- [5] X. Li, J. He, P. Vijayakumar, X. Zhang, and V. Chang, “A verifiable privacy-preserving machine learning prediction scheme for edge-enhanced hepss,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 8, pp. 5494–5503, 2021.
- [6] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” in *Proceedings of the seventeenth annual ACM symposium on Theory of computing*. ACM, 1985, pp. 291–304.
- [7] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun, “Scaling up trustless dnn inference with zero-knowledge proofs,” *arXiv preprint arXiv:2210.08674*, 2022.
- [8] Z. Xing, Z. Zhang, J. Liu, Z. Zhang, M. Li, L. Zhu, and G. Russello, “Zero-knowledge proof meets machine learning in verifiability: A survey,” *arXiv preprint arXiv:2310.14848*, 2023.
- [9] Z. Peng, T. Wang, C. Zhao, G. Liao, Z. Lin, Y. Liu, B. Cao, L. Shi, Q. Yang, and S. Zhang, “A survey of zero-knowledge proof based verifiable machine learning,” *arXiv preprint arXiv:2502.18535*, 2025.
- [10] T. Liu, X. Xie, and Y. Zhang, “Zkcnv: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2968–2985.
- [11] K. Abbaszadeh, C. Pappas, J. Katz, and D. Papadopoulos, “Zero-knowledge proofs of training for deep neural networks,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4316–4330.
- [12] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, “Delegating computation: interactive proofs for muggles,” *Journal of the ACM (JACM)*, vol. 62, no. 4, pp. 1–64, 2015.
- [13] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT press, 2016.

- [14] Z. C. Lipton, J. Berkowitz, and C. Elkan, “A critical review of recurrent neural networks for sequence learning,” *arXiv preprint arXiv:1506.00019*, 2015.
- [15] T. Xie, Y. Zhang, and D. Song, “Orion: Zero knowledge proof with linear prover time,” in *Annual International Cryptology Conference*. Springer, 2022, pp. 299–328.
- [16] J. Thaler, “Time-optimal interactive proofs for circuit evaluation,” in *Annual Cryptology Conference*. Springer, 2013, pp. 71–89.
- [17] Q. Dao and J. Thaler, “More optimizations to sum-check proving,” *Cryptology ePrint Archive*, 2024.
- [18] M. Albrecht *et al.*, “Mimc: Efficient encryption and cryptographic hashing with minimal multiplicative complexity,” in *ASIACRYPT*, 2016.
- [19] G. Cormode, J. Thaler, and K. Yi, “Verifying computations with streaming interactive proofs,” *arXiv preprint arXiv:1109.6882*, 2011.
- [20] S. Lee, H. Ko, J. Kim, and H. Oh, “vcnn: Verifiable convolutional neural network based on zk-snarks,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4254–4270, 2024.
- [21] A. E. Kosba, D. Papadopoulos, C. Papamanthou, M. F. Sayed, E. Shi, and N. Triandopoulos, “{TRUESET}: Faster {Verifiable} set computations,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 765–780.
- [22] R. E. Burkard, “Quadratic assignment problems,” *European Journal of Operational Research*, vol. 15, no. 3, pp. 283–289, 1984.
- [23] M. Campanelli, D. Fiore, and A. Querol, “Legosnark: Modular design and composition of succinct zero-knowledge proofs,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 2075–2092.
- [24] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, “Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 501–518.
- [25] K. Yang, P. Sarkar, C. Weng, and X. Wang, “Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2986–3001.
- [26] T. Lu, H. Wang, W. Qu, Z. Wang, J. He, T. Tao, W. Chen, and J. Zhang, “An efficient and extensible zero-knowledge proof framework for neural networks,” *Cryptology ePrint Archive*, 2024.
- [27] P. Schoppmann, A. Gascón, L. Reichert, and M. Raykova, “Distributed vector-ole: Improved constructions and implementation,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1055–1072.
- [28] M. Hao, H. Chen, H. Li, C. Weng, Y. Zhang, H. Yang, and T. Zhang, “Scalable zero-knowledge proofs for non-linear functions in machine learning,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 3819–3836.
- [29] Y. Li and X. Fan, “Summer: Recursive zero-knowledge proofs for scalable rnn training,” *Cryptology ePrint Archive*, 2025.
- [30] P. Valiant, “Incrementally verifiable computation or proofs of knowledge imply time/space efficiency,” in *Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19–21, 2008. Proceedings 5*. Springer, 2008, pp. 1–18.
- [31] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, “Algebraic methods for interactive proof systems,” *Journal of the ACM (JACM)*, vol. 39, no. 4, pp. 859–868, 1992.
- [32] A. Chiesa, M. A. Forbes, and N. Spooner, “A zero knowledge sumcheck and its applications,” *arXiv preprint arXiv:1704.02086*, 2017.
- [33] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, “Marlin: Preprocessing zksnarks with universal and updatable srs,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2020, pp. 738–768.
- [34] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Conference on the theory and application of cryptographic techniques*. Springer, 1986, pp. 186–194.
- [35] J. L. Elman, “Finding structure in time,” *Cognitive science*, vol. 14, no. 2, pp. 179–211, 1990.
- [36] S. Garg, A. Goel, S. Jha, S. Mahloujifar, M. Mahmoody, G.-V. Policharla, and M. Wang, “Experimenting with zero-knowledge proofs of training,” in *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*, 2023, pp. 1880–1894.
- [37] P. Mohassel and P. Rindal, “Aby3: A mixed protocol framework for machine learning,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 35–52.
- [38] G. C. Cardarilli, L. Di Nunzio, R. Fazzolari, D. Giardino, A. Nannarelli, M. Re, and S. Spanò, “A pseudo-softmax function for hardware-based high speed image classification,” *Scientific reports*, vol. 11, no. 1, p. 15307, 2021.
- [39] A. Epasto, M. Mahdian, V. Mirrokni, and E. Zampetakis, “Optimal approximation-smoothness tradeoffs for soft-max functions,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 2651–2660, 2020.
- [40] J. Zhang, T. Liu, W. Wang, Y. Zhang, D. Song, X. Xie, and Y. Zhang, “Doubly efficient interactive proofs for general arithmetic circuits with linear prover time,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 159–177.
- [41] B. Bünz, P. Mishra, W. Nguyen, and W. Wang, “Accumulation without homomorphism,” *Cryptology ePrint Archive*, 2024.
- [42] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, “Recursive composition and bootstrapping for snarks and proof-carrying data,” in *STOC*, 2013.
- [43] G. Fenzi and Y. Zhang, “zip: Reducing proof sizes for hash-based snarks,” *Cryptology ePrint Archive*, 2025.
- [44] T. Xie, Y. Zhang, and D. Song, “Orion: Zero knowledge proof with linear prover time,” *Cryptology ePrint Archive*, Paper 2022/1010, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1010>
- [45] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Conference on the theory and application of cryptographic techniques*. Springer, 1987, pp. 369–378.
- [46] “zkRNN: Zero-knowledge proofs for recurrent neural network inference (anonymous repository),” <https://anonymous.4open.science/r/zkRNN-5CA8/>, 2026.
- [47] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [48] F. A. Gers, J. Schmidhuber, and F. Cummins, “Learning to forget: Continual prediction with lstm,” *Neural computation*, vol. 12, no. 10, pp. 2451–2471, 2000.
- [49] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.

APPENDIX

A. Definitions

Definition 2 (Zero Knowledge Recurrent Neural Network (zkRNN)). *We say that a scheme zkRNN = (KeyGen, Com, Prove, Verify) is a zero-knowledge proof of recurrent neural network inference if the following holds.*

Let λ be the security parameter and $\mathcal{C}$ be the public RNN-step function $(a_k, y_k) = \mathcal{C}(W, a_{k-1}, x_k)$.

Completeness. *Given an RNN model W , with commitment $\sigma_W \leftarrow \text{Com}(W, \text{pp})$, an initial state commitment σ_{a_0} , a valid witness a_{i-1} and proof π_{i-1} for step $i-1$ (with commitment $\sigma_{a_{i-1}}$), and a new public input x_i , the following equals to 1:*

$$\Pr \left[b = 1 \mid \begin{array}{l} (a_i, y_i) \leftarrow \mathcal{C}(W, a_{i-1}, x_i); \\ (\sigma_{a_i}, \pi_i) \leftarrow \text{Prove}(i, \sigma_W, \sigma_{a_0}, a_{i-1}, \sigma_{a_{i-1}}, \pi_{i-1}, x_i, y_i, \text{pp}); \\ b \leftarrow \text{Verify}(i, \sigma_W, \sigma_{a_0}, \sigma_{a_i}, y_i, \pi_i, \text{pp}) \end{array} \right]$$

Knowledge Soundness. *For any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}^{\mathcal{A}}$ such that the following holds:*

$$\Pr \left[\begin{array}{l} (W, \{a_k\}_{k=0}^i) \leftarrow \mathcal{E}^{\mathcal{A}}(\text{pp}) : \\ \forall k \in \{1, \dots, i\} : (a_k, y_k) = \mathcal{C}(W, a_{k-1}, x_k); \\ \sigma_W^* = \text{Com}(W, \text{pp}); \sigma_{a_0}^* = \text{Com}(a_0, \text{pp}); \sigma_{a_i}^* = \text{Com}(a_i, \text{pp}) \end{array} \right] \\ \geq \Pr \left[\begin{array}{l} (i, \sigma_W^*, \sigma_{a_0}^*, \sigma_{a_i}^*, \{x_k\}_{k=1}^i, \{y_k\}_{k=1}^i, \{\pi_k^*\}_{k=1}^i) \leftarrow \mathcal{A}(\text{pp}) : \\ \text{Verify}(i, \sigma_W^*, \sigma_{a_0}^*, \sigma_{a_i}^*, \{x_k\}_{k=1}^i, \{y_k\}_{k=1}^i, \{\pi_k^*\}_{k=1}^i, \text{pp}) = 1 \end{array} \right] - \text{negl}(\lambda)$$

Zero Knowledge. *For any PPT algorithm $\mathcal{A}$ and all RNN models W , initial states a_0 , and input sequences $\{x_k\}_{k=1}^i$, there exists a simulator $\mathcal{S}$ (given oracle access to $\mathcal{C}$) such that*

the following two experiments are computationally indistinguishable:

Real_{A,W}(i, pp):

- 1) $\sigma_W \leftarrow \text{Com}(W), \sigma_{a_0} \leftarrow \text{Com}(a_0)$.
- 2) For $k = 1 \dots i$:
 $(a_k, y_k) = \mathcal{C}(W, a_{k-1}, x_k)$
 $(\sigma_{a_k}, \pi_k) \leftarrow \text{Prove}(k, \sigma_W, \sigma_{a_0}, a_{k-1}, \sigma_{a_{k-1}}, \pi_{k-1}, x_k, \text{pp})$
- 3) $b \leftarrow \mathcal{A}(\text{pp}, \sigma_W, \sigma_{a_0}, \sigma_{a_i}, \{x_k\}_{k=1}^i, \{y_k\}_{k=1}^i, \pi_i)$
- 4) Output b

Ideal_{A,S}(i, pp):

- 1) $(\sigma_W^*, \sigma_{a_0}^*, \sigma_{a_i}^*, \pi_i^*) \leftarrow \mathcal{S}(i, \text{pp}, \{x_k\}_{k=1}^i)$, given oracle access to $\mathcal{C}$
- 2) $b \leftarrow \mathcal{A}(\text{pp}, \sigma_W^*, \sigma_{a_0}^*, \sigma_{a_i}^*, \{x_k\}_{k=1}^i, \{y_k\}_{k=1}^i, \pi_i^*)$
- 3) Output b

We require that:

$$|\Pr[\text{Real}_{A,W} = 1] - \Pr[\text{Ideal}_{A,S} = 1]| \leq \text{negl}(\lambda)$$

B. Protocols

Protocol Sumcheck

- 1) In the first round, P sends the polynomial

$$f_1(x_1) := \sum_{b_2, \dots, b_\ell \in \{0,1\}} f(x_1, b_2, b_3, \dots, b_\ell)$$

V checks if $H = f_1(0) + f_1(1)$ and sends $r_1 \leftarrow \mathbb{F}$.

- 2) In the i th round ($2 \leq i \leq \ell - 1$), P sends the polynomial

$$f_i(x_i) := \sum_{b_{i+1}, \dots, b_\ell \in \{0,1\}} f(r_1, \dots, r_{i-1}, x_i, b_{i+1}, \dots, b_\ell)$$

V checks if $f_{i-1}(r_{i-1}) = f_i(0) + f_i(1)$ and sends $r_i \leftarrow \mathbb{F}$.

- 3) In the ℓ th round, P sends the polynomial

$$f_\ell(x_\ell) = f(r_1, r_2, \dots, r_{\ell-1}, x_\ell)$$

V checks if $f_{\ell-1}(r_{\ell-1}) = f_\ell(0) + f_\ell(1)$. The verifier samples the final random challenge $r_\ell \in \mathbb{F}$. Given oracle access to f , V accepts if and only if $f_\ell(r_\ell) = f(r_1, r_2, \dots, r_\ell)$.

Fig. 5. Protocol Sumcheck

Theorem A.1. [31] Given a finite field $\mathbb{F}$ and an ℓ -variate polynomial $f : \mathbb{F}^\ell \rightarrow \mathbb{F}$ with individual degree at most d , Protocol sumcheck (cf. Fig. 5) is an interactive proof for the statement $H = \sum_{x \in \{0,1\}^\ell} f(x)$ with soundness error at most $O(\frac{d\ell}{|\mathbb{F}|})$. It uses ℓ rounds of interaction, verifier running time $O(d\ell)$ plus one oracle query to f , and prover running time $O(2^\ell)$.

Theorem A.2. [12] Let $\mathcal{C} : \mathbb{F}^{s_{in}} \rightarrow \mathbb{F}^{s_{out}}$ be a layered arithmetic circuit of depth d and size $|\mathcal{C}|$. The GKR protocol 6 is an interactive proof for the evaluation of $\mathcal{C}$ with soundness error $O(\frac{d \cdot \log |\mathcal{C}|}{|\mathbb{F}|})$. The protocol requires $O(d \cdot \log |\mathcal{C}|)$ rounds of interaction. The prover runs in time $O(|\mathcal{C}|)$, and the verifier

runs in time $O(s_{in} + s_{out} + d \cdot \log |\mathcal{C}| + T)$, where T is the time to evaluate the circuit's wiring predicates.

Theorem A.3. [11] The aggregation scheme presented in Protocol 7 satisfies completeness, knowledge soundness, and zero knowledge.

C. RNN different variants

a) **Long Short-Term Memory (LSTM):** Long Short-Term Memory (LSTM) networks address the vanishing gradient problem of standard RNNs by introducing a memory cell containing a Constant Error Carousel (CEC), which maintains error flow over long time lags [47]. To prevent the internal state from growing indefinitely during continual processing, [48] introduced the *forget gate*, which allows the network to adaptively reset its memory resources when stored information becomes obsolete. The cell state c_t and hidden state h_t are updated as follows:

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ h_t &= o_t \odot \tanh(c_t), \end{aligned}$$

where σ denotes the logistic sigmoid function, $\odot$ represents element-wise multiplication, and W and U denote the input and recurrent weight matrices, respectively. The forget gate $f_t \in [0, 1]$ scales the previous cell state, enabling the network to learn both long-term dependencies and local state resets [48].

Gated Recurrent Unit (GRU) The Gated Recurrent Unit (GRU) simplifies the LSTM architecture by merging the cell and hidden states into a single activation while retaining the ability to capture long-term dependencies through gating [49]. The GRU employs two multiplicative gates: a *reset gate* r_t , which determines whether to ignore the previous hidden state, and an *update gate* z_t , which controls how much of the previous state is carried over to the current step. The hidden state h_t is computed as:

$$\begin{aligned} r_t &= \sigma(W_r x_t + U_r h_{t-1}), \\ z_t &= \sigma(W_z x_t + U_z h_{t-1}), \\ \tilde{h}_t &= \tanh(W x_t + U(r_t \odot h_{t-1})), \\ h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t, \end{aligned}$$

where σ is the logistic sigmoid function and $\odot$ denotes element-wise multiplication. By regulating the information flow via z_t and r_t , the GRU adaptively remembers or forgets information without the separate memory cell found in LSTMs [49].

D. A Recursive Framework of RNN Inference

Modern recurrent models share the same high-level structure, a small state that evolves over time under a fixed set of

Protocol GKR

Let $\mathbb{F}$ be a finite field and $C : \mathbb{F}^{s_{in}} \rightarrow \mathbb{F}^{s_{out}}$ be a layered arithmetic circuit of depth d . P wants to convince V that $C(\text{in}) = \text{out}$, where in is the input given by V and out is the output sent to V. Without loss of generality, we pad s_{in} and s_{out} to powers of 2. The protocol proceeds as follows.

- 1) V picks a random $z \in \mathbb{F}$ and sends it to P. Both parties P and V compute $\tilde{V}_0(z)$, where $\tilde{V}_0$ is the multilinear extension of array out.
- 2) P and V invoke a sumcheck protocol [17] on

$$\tilde{V}_0(z) = \sum_{x, y \in \{0,1\}^{s_1}} \tilde{add}_0(z, x, y) \cdot (\tilde{V}_1(x) + \tilde{V}_1(y)) + \tilde{mult}_0(z, x, y) \cdot \tilde{V}_1(x) \cdot \tilde{V}_1(y)$$

At the end of the protocol, V receives two evaluations $\tilde{V}_1(u^{(1)}), \tilde{V}_1(v^{(1)})$. V computes $\tilde{add}_0(z, u^{(1)}, v^{(1)})$ and $\tilde{mult}_0(z, u^{(1)}, v^{(1)})$ and checks that $\tilde{add}_0(z, u^{(1)}, v^{(1)}) \cdot (\tilde{V}_1(u^{(1)}) + \tilde{V}_1(v^{(1)})) + \tilde{mult}_0(z, u^{(1)}, v^{(1)}) \cdot \tilde{V}_1(u^{(1)}) \cdot \tilde{V}_1(v^{(1)})$ equals to the last message of sumcheck.

- 3) For $i = 1, 2, \dots, d$:

- V picks two random $\alpha^{(i)}$ and $\beta^{(i)}$ and send them to P.
- P and V invoke a sumcheck protocol on

$$\begin{aligned} \alpha^{(i)} \tilde{V}_i(u^{(i)}) + \beta^{(i)} \tilde{V}_i(v^{(i)}) = & \sum_{x, y \in \{0,1\}^{s_{i+1}}} (\alpha^{(i)} \tilde{add}_i(u^{(i)}, x, y) + \beta^{(i)} \tilde{add}_i(v^{(i)}, x, y)) \cdot (\tilde{V}_{i+1}(x) + \tilde{V}_{i+1}(y)) \\ & + (\alpha^{(i)} \tilde{mult}_i(u^{(i)}, x, y) + \beta^{(i)} \tilde{mult}_i(v^{(i)}, x, y)) \cdot \tilde{V}_{i+1}(x) \cdot \tilde{V}_{i+1}(y) \end{aligned}$$

At the end of the protocol, V receives two evaluations $\tilde{V}_{i+1}(u^{(i+1)}), \tilde{V}_{i+1}(v^{(i+1)})$. V computes the following and if it does not equal to the last message of the sumcheck, returns 0.

$$\begin{aligned} & (\alpha^{(i)} \tilde{add}_i(u^{(i)}, u^{(i+1)}, v^{(i+1)}) + \beta^{(i)} \tilde{add}_i(v^{(i)}, u^{(i+1)}, v^{(i+1)})) \cdot (\tilde{V}_{i+1}(u^{(i+1)}) + \tilde{V}_{i+1}(v^{(i+1)})) \\ & + (\alpha^{(i)} \tilde{mult}_i(u^{(i)}, u^{(i+1)}, v^{(i+1)}) + \beta^{(i)} \tilde{mult}_i(v^{(i)}, u^{(i+1)}, v^{(i+1)})) \cdot \tilde{V}_{i+1}(u^{(i+1)}) \cdot \tilde{V}_{i+1}(v^{(i+1)}) \end{aligned}$$

- 4) At the input layer d , V has two claims $\tilde{V}_d(u^{(d)})$ and $\tilde{V}_d(v^{(d)})$. As V has access to the input in, it is possible to directly compute $\tilde{V}_d$ and evaluate it at two points u^d and v^d . If claims are correct, V returns 1, otherwise returns 0.

Fig. 6. Protocol GKR

Protocol Polynomial Commitment Aggregation

Parameters: Let $\text{PCS} = (\text{KeyGen}, \text{Commit}, \text{Open}, \text{Verify})$ be the Orion [15] polynomial commitment scheme with public parameters pp . Let $f_1, f_2, \dots, f_k$ be ℓ -variate polynomials with variable-degree d , each has an evaluation $y_i = f_i(x_i)$ and committed to by σ_i with randomness r_i . M_i denotes coefficients of f_i parsed as a square matrix and $C_{i,1}, C_{i,2}$ denote the associated encoded matrices generated during the Orion commitment procedure (cf. Section 5.2).

Phase 1: Evaluation Aggregation

- 1) P and V interpolate a polynomial $L : \mathbb{F} \rightarrow \mathbb{F}^\ell$ such that $L(i) = x_i$.
- 2) $\forall i = 1$ to k , P samples random univariate polynomials $h_{i,1}, \dots, h_{i,\ell}$, each of degree $d\ell$, sets $h_i(z_1, \dots, z_\ell) \leftarrow h_{i,1}(z_1) + \dots + h_{i,\ell}(z_\ell)$, and sends $v_i \leftarrow h_i(x_i)$ and $\sigma'_i \leftarrow \text{PCS.Commit}(h_i, r'_i, \text{pp})$. M'_i denotes the coefficient matrix of h_i and $C'_{i,1}, C'_{i,2}$ denote the associated encoded matrices.
- 3) $\forall i = 1$ to k , V responds with a random challenge β_i .
- 4) $\forall i = 1$ to k , P sends the polynomial $g_i \leftarrow (f_i + \beta_i h_i) \circ L$.
- 5) $\forall i = 1$ to k , V checks whether $g_i(i) = y_i + \beta_i v_i$ and responds with random challenges α_i and r .
- 6) Parties compute $g \leftarrow \sum_{i=1}^k \alpha_i g_i$ and return $x^* = L(r)$, and $y^* = g(r)$ as output evaluation points.
- 7) P computes the output witness polynomial $f^* = \sum_{i=1}^k \alpha_i f_i + \alpha_i \beta_i h_i$.

Phase 2: Commitment Aggregation

- 1) P sends $\sigma^* \leftarrow \text{PCS.Commit}(f^*, r^*, \text{pp})$. M^* denotes the coefficient matrix of f^* and C_1^*, C_2^* denote the associated encoded matrices.
- 2) V randomly samples and sends a subset of indices I of size $t \in \Theta(\lambda)$.
- 3) $\forall i = 1$ to $k, \forall \text{idx} \in I$, P sends $s^* = C_2^*[\text{idx}]$, $s_i = C_{i,2}[\text{idx}]$, and $s'_i = C'_{i,2}[\text{idx}]$ with their corresponding Merkle proofs.
- 4) $\forall i = 1$ to $k, \forall \text{idx} \in I$, V checks whether $s^* = \sum_{i=1}^k \alpha_i s_i + \alpha_i \beta_i s'_i$ and the received Merkle proofs are valid.
- 5) The verifier returns (σ^*, x^*, y^*) as the aggregate instances, the prover returns $(\sigma^*, x^*, y^*), (f^*, r^*)$ as the aggregated instance/witness tuple.

Fig. 7. Polynomial Commitment Aggregation

parameters, but they differ in *what* is carried across steps and *how* each step updates it. A vanilla RNN maintains a single hidden vector, a GRU keeps one history value with additional gating intermediates, and an LSTM maintains multiple persistent values (typically a hidden state and a cell state). Protocol C, presented in Fig 8, is a single streaming-and-recursive framework that can be instantiated with different recurrent architectures by swapping only the step circuit and its step-proof, while keeping the recursive composition mechanism.

Protocol C models one time step as a *generic* circuit C that takes the public input x_i , the previous recurrent state bundle $s_{i-1} = (s_{i-1}^{(1)}, \dots, s_{i-1}^{(r)})$, and the committed model parameters W , and outputs the next bundle s_i , the public output y_i , and auxiliary witness aux_i . s_i can contain one persistent value (as in GRUs) or multiple (as in LSTMs), without changing any downstream logic. On each incoming stream element, **Prove-Leaf** runs C according to the model to compute (s_i, y_i, aux_i) , commits to the new state (and to any auxiliary polynomials required by the chosen step-proof), and then produces a *step transcript proof* together with commitment–evaluation claims $\mathcal{E}_i^{\text{raw}}$ and wiring/evaluation claims $\mathcal{W}_i^{\text{raw}}$.

The step-proof itself follows the same modular logic as Protocol A, but parameterized by C . Linear subcomputations inside C are reduced to canonical matrix multiplication relations and proven via a time-optimal matrix-multiplication sumcheck (e.g., Thaler-style SumMatMul). Non-linear subcomputations (activations, comparisons, exponentiation approximations, gating, etc.) are handled by quantization and a GKR-style circuit, with any required bit-decomposition or range/consistency constraints proven using dedicated sumcheck sub-protocols rather than inflating the main circuit. In other words, the framework only requires C to be decomposed into “linear parts” handled by matrix-multiplication sumcheck and “non-linear parts” handled by GKR plus succinct consistency checks.

To keep proofs and verification time from growing linearly with the stream length, Protocol C defers expensive polynomial-opening checks by *folding* the per-step evaluation claims into accumulators. After producing $(\pi_{i,G}, \mathcal{E}_i^{\text{raw}}, \mathcal{W}_i^{\text{raw}})$, the prover aggregates $\mathcal{E}_i^{\text{raw}}$ (per polynomial family) into a data accumulator agg_i with an aggregation proof and a carried witness, and compresses $\mathcal{W}_i^{\text{raw}}$ into a wiring accumulator wp_i via evaluation reduction. The leaf proof package therefore contains the arithmetic transcript component $\pi_{i,G}$, the aggregated data component $(\text{agg}_i, \pi_{i,A}, \pi_{i,E})$, and the aggregated wiring component wp_i .

Recursive composition is then performed over a k -ary tree to control recursion depth and avoid very deep sequential folding. Leaves correspond to individual time steps, and as the stream progresses the system eagerly merges groups of k child packages into a parent package. In each merge an augmented function F_A is defined that (re)derives the Fiat–Shamir challenges and hash computations and then *verifies the child packages*: for each child, F_A checks the child transcript $\pi_{j,G}$ using the sumcheck and GKR verifiers, and checks the child aggregation proof $\pi_{j,A}$ to ensure the child’s output accumulator correctly folds its inputs together with the new

evaluation claims implied by that transcript. Crucially, F_A does *not* verify polynomial openings; instead it produces the next layer’s implied claims and propagates accumulators upward. The parent then proves, via its own transcript $\pi_{\text{parent},G}$, that it executed F_A correctly, and it folds the children’s data and wiring accumulators into $(\text{agg}_{\text{parent}}, \text{wp}_{\text{parent}})$. Using arity $k \geq 2$ yields recursion depth $O(\log_k T)$, while preserving the same leaf semantics and streaming behavior.

Verification mirrors the root logic of the original zkRNN design but remains agnostic to the choice of recurrent architecture. The verifier decompresses the transmitted proof (if compression is used), recomputes Fiat–Shamir challenges and hashes, and verifies the root transcript π_G , which certifies that the root executed F_A correctly and therefore that all in-circuit child verifications and accumulator-validations were performed throughout the tree. It then checks the final wiring accumulator against the known wiring predicates, verifies the aggregation proof π_A showing that the root data accumulator correctly summarizes all deferred commitment–evaluation claims, and checks the aggregated public-input claim against the verifier’s locally maintained stream aggregate, and finally performs a single polynomial-opening verification PCS.Verify for the root aggregate (or checks the deferred witness if the opening is postponed to a higher layer). Altogether, this reduces verifier work from “verify every step” to “verify one root package”, while keeping the framework flexible enough to plug in different recurrent step circuits C (vanilla RNN, GRU, LSTM, and beyond) without changing the recursion mechanism.

E. Security Proofs

1) *Proof of Theorem III.1*: By completeness of GKR, SumMatMul and the completeness of the polynomial commitment scheme PCS, completeness of Protocol A (cf. Fig. 2) is immediate.

We now show a negligible soundness error by a phase-by-phase analysis. Suppose the prover $\mathcal{P}^*$ is dishonest and sends some $y_i^* \neq y_i$.

Phase 1: Following the Schwartz–Zippel lemma and the soundness of GKR, we have $\tilde{y}_i^*(r_y) \neq \tilde{y}_i(r_y)$ with high probability unless: (i) $\tilde{s}_i^*(r_{s_i}) \neq \tilde{s}_i(r_{s_i})$, or (ii) $\widetilde{\text{AUX}}_i^*(r_{\text{AUX},1}) \neq \widetilde{\text{AUX}}_i(r_{\text{AUX},1})$. In case (i), the parties run SumMatMul on $s_i = W_{ya}a_i + b_y$. By the soundness of SumMatMul, we have, with high probability, that at least one of

$$\begin{aligned} \tilde{a}_i^*(r_{a,\text{out}}) &\neq \tilde{a}_i(r_{a,\text{out}}), \\ \tilde{W}_{ya}^*(r_{W_{ya}}) &\neq \tilde{W}_{ya}(r_{W_{ya}}), \tilde{b}_y^*(r_{b_y}) \neq \tilde{b}_y(r_{b_y}) \end{aligned}$$

occurs. A mismatch in $\tilde{a}_i$ proceeds to Phase 2, while mismatches in $\tilde{W}_{ya}$ or $\tilde{b}_y$ are mismatches on committed polynomials and proceed to Phase 4. In case (ii) a mismatch on the auxiliary polynomial at point $r_{\text{AUX},1}$ also proceed to Phase 4.

Phase 2: Suppose that case (i) in Phase 1 occurs and there exists at least one instance where $\tilde{a}_i^*(r_{a,\text{out}}) \neq \tilde{a}_i(r_{a,\text{out}})$. By the soundness of GKR on the relation $a_i = F_1(z_i)$, we have, with high probability, either: (i) $\tilde{z}_i^*(r_{z_i}) \neq \tilde{z}_i(r_{z_i})$, or

Protocol C (Framework): Streaming and Recursive ZK Proof of Recurrent Inference for a Generic Step Circuit C

Parameters. Let SumMatMul be the time-optimal sumcheck protocol for matrix multiplication. Let GKR be a generic GKR-style zero-knowledge proof system. Let PCS = (KeyGen, Commit, Open, Verify) be a polynomial commitment scheme. Let AGG = (Prove, Verify) be a non-interactive aggregation scheme that supports (i) aggregation of evaluation claims per polynomial family, and (ii) aggregation of public input streams (e.g., via interpolation). Let (Compress, Decompress) be a lossless compressor. Let $k \geq 2$ be the recursion arity. Let π_0 be a trivially satisfying proof package.

Model and recurrent state. Model commitments σ_W commit to all parameters required by C (weights, biases, etc.). The recurrent state at time t is a *bundle* $\mathbf{s}_t = (s_t^{(1)}, \dots, s_t^{(r)})$ of persistent hidden value(s). Let σ_{s_0} be the commitment(s) to the initial state bundle.

Goal. Process an input stream $\{x_i\}_{i=1}^T$ and return outputs $\{y_i\}_{i=1}^T$ immediately, while maintaining a buffer of proof packages that can be merged into a single succinct root proof on demand.

Streaming Orchestration Maintain the current committed state bundle $(\mathbf{s}, \sigma_{\mathbf{s}})$ and buffers of pending proof packages per recursion level.

OnInput(x_i): Generate a leaf proof π_{leaf} and output y_i via ProveLeaf, return y_i immediately, store the updated state $(\mathbf{s}_i, \sigma_{\mathbf{s}_i})$, and buffer π_{leaf} (eagerly merging via ProveInternalNode whenever a level accumulates k proofs).

FinalizeProof(): Recursively merge all buffered proofs up to the root via ProveInternalNode (padding if needed), obtain π_{root} , and output $\pi_{\text{zip}} \leftarrow \text{Compress}(\pi_{\text{root}})$.

ProveLeaf($i, x_i, \sigma_W, \mathbf{s}_{i-1}, \sigma_{\mathbf{s}_{i-1}}, pp$): Generates a leaf proof for one step of the chosen recurrent architecture.

- 1) **Run the step circuit C :** Compute $(\mathbf{s}_i, y_i, \text{aux}_i) \leftarrow C(x_i, \mathbf{s}_{i-1}; W)$.
- 2) **Commit updated state:** Compute commitment(s) $\sigma_{\mathbf{s}_i} \leftarrow \text{PCS.Commit}(\mathbf{s}_i, r_{\mathbf{s}_i}, pp)$. If the step-proof requires auxiliary witness polynomials, commit to them as $\sigma_{\text{aux}_i} \leftarrow \text{PCS.Commit}(\text{aux}_i, r_{\text{aux}_i}, pp)$.
- 3) **Generate step proof:** Produce the transcript proof and the raw claims $(\pi_{i,G}, \mathcal{E}_i^{\text{raw}}, \mathcal{W}_i^{\text{raw}}) \leftarrow \text{ProofOfStep}_C.\text{Prove}(x_i, y_i, \sigma_W, \sigma_{\mathbf{s}_{i-1}}, \sigma_{\mathbf{s}_i}, \sigma_{\text{aux}_i})$. Here, ProofOfStep_C is instantiated as follows: (i) handle *linear* components of C via reductions to canonical matrix multiplication and prove them with SumMatMul; (ii) handle *non-linearities* via fixed-point/quantization plus a GKR circuit (with the necessary consistency checks proved via sumcheck sub-protocols).
- 4) **Aggregate leaf claims:** If needed, run AGG.Prove on $\mathcal{E}_i^{\text{raw}}$ *per polynomial family* to obtain the leaf data accumulator(s) and their proof/witness. Compress wiring claims $\mathcal{W}_i^{\text{raw}}$ into a wiring accumulator wp_i via evaluation reduction.
- 5) **Assemble:** Output the leaf proof package $\pi_{\text{leaf}} := (\pi_{i,G}, \pi_{i,A}, \pi_{i,E}, \emptyset, \text{agg}_i, \emptyset, \text{wp}_i)$, together with $(\sigma_{\mathbf{s}_i}, y_i)$.

ProveInternalNode($\{\pi_{\text{child}_j}\}_{j=1}^k, pp$):

- 1) **Define F_A .** On input $\{\pi_{\text{child}_j}\}_{j=1}^k$ and required public data:
 - a) Recompute Fiat–Shamir challenges / hashes needed to verify each child.
 - b) For each $j \in [1, k]$:
 - i) Parse $\pi_{\text{child}_j} = (\pi_{j,G}, \pi_{j,A}, \pi_{j,E}, \{\text{agg}_{j,\text{in},m}\}_{m=1}^k, \text{agg}_j, \{\text{wp}_{j,\text{in},m}\}_{m=1}^k, \text{wp}_j)$.
 - ii) Run the verifier for $\pi_{j,G}$ (e.g., GKR.V/SumMatMul.V) using $\{\text{agg}_{j,\text{in},m}\}_{m=1}^k, \{\text{wp}_{j,\text{in},m}\}_{m=1}^k$ and other public data. If it accepts, obtain $\mathcal{E}_j^{\text{new}}$ and wp_j .
 - iii) Run AGG.V on $\pi_{j,A}$ to check that agg_j aggregates $\{\text{agg}_{j,\text{in},m}\}_{m=1}^k$ and $\mathcal{E}_j^{\text{new}}$.
 - c) If all sumchecks (a) and aggregations (b) pass, the F_A computation is valid.
- 2) Set $u_{\text{parent}} := \sigma_{a_{s_0}} \parallel \sigma_{s_{\text{child}_1}} \parallel \dots \parallel \sigma_{s_{\text{child}_k}} \parallel \pi_{\text{child}_1} \parallel \dots \parallel \pi_{\text{child}_k}$ and commit $\sigma_{u_{\text{parent}}} \leftarrow \text{PCS.Commit}(u_{\text{parent}}, r_{u_{\text{parent}}}, pp)$.
- 3) Generate $\pi_{\text{parent},G}$ proving correct execution of F_A , including the hash/challenge computations. In the same proof, fold $\{\text{wp}_j\}_{j=1}^k$ into $\text{wp}_{\text{parent}}$ via evaluation-reduction sumcheck, and similarly fold the evaluations of $\tilde{u}_{\text{parent}}$ as required.
- 4) Compute the parent data aggregate: $(\text{agg}_{\text{parent}}, \pi_{\text{parent},A}) \leftarrow \text{AGG.P}(\{\text{agg}_j, \pi_{j,E}\}_{j=1}^k, (\text{remaining commitment–evaluation claims}))$, and also (if supported) aggregate the corresponding public inputs for these k children.
- 5) Set $\pi_{\text{parent},E}$ to PCS.Open($\text{agg}_{\text{parent}}$) if this is the root; otherwise set it to the deferred opening witness for $\text{agg}_{\text{parent}}$.
- 6) Return $\pi_{\text{parent}} := (\pi_{\text{parent},G}, \pi_{\text{parent},A}, \pi_{\text{parent},E}, \{\text{agg}_j\}_{j=1}^k, \text{agg}_{\text{parent}}, \{\text{wp}_j\}_{j=1}^k, \text{wp}_{\text{parent}})$.

Verify($t, \sigma_W, \sigma_{s_0}, \sigma_{\mathbf{s}_t}, \pi_{\text{zip}}, pp$):

- 1) Decompress $\pi_{\text{zip}} \rightarrow \pi_{\text{root}}$ and parse $\pi_{\text{root}} = (\pi_G, \pi_A, \pi_E, \{\text{agg}_{\text{in},m}\}_{m=1}^k, \text{agg}_{\text{root}}, \{\text{wp}_{\text{in},m}\}_{m=1}^k, \text{wp}_{\text{root}})$.
- 2) Recompute all Fiat–Shamir challenges / hashes needed for π_G, π_A .
- 3) Run GKR.V/SumMatMul.V on π_G using public inputs $(t, \sigma_W, \sigma_{s_0}, \sigma_{\mathbf{s}_t}, \{x_i\}_{i=1}^n)$ and child inputs $\{\text{agg}_{\text{in},m}\}_{m=1}^k, \{\text{wp}_{\text{in},m}\}_{m=1}^k$ and obtain $(\mathcal{E}^{\text{root}}, \text{wp}_{\text{root}})$.
- 4) Check wp_{root} against the known wiring predicates.
- 5) Verify π_A to check agg_{root} aggregates $\{\text{agg}_{\text{in},m}\}_{m=1}^k$ and $\mathcal{E}^{\text{root}}$ (and any additional evaluation claims output by the root verification), and verify the aggregated public-input claim for the covered segment against the locally computed one.
- 6) If final, run PCS.Verify($\text{agg}_{\text{root}}, \pi_E$); else check π_E is a valid deferred opening witness for agg_{root} .

Fig. 8. A generic framework parameterized by a per-step circuit C ; correspond to different choices of C (vanilla RNN / GRU / LSTM) and ProofOfStep $_C$.

(ii) $\widetilde{\text{AUX}}_i^*(r_{\text{AUX},2}) \neq \widetilde{\text{AUX}}_i(r_{\text{AUX},2})$. In case (i), the parties run SumMatMul on $z_i = W_{ax}x_i + W_{aa}a_{i-1} + b_a$. By the soundness of SumMatMul, with high probability at least one of

$$\begin{aligned} \tilde{x}_i^*(r_{x_i}) &\neq \tilde{x}_i(r_{x_i}), \tilde{a}_{i-1}^*(r_{a_{i-1}}) \neq \tilde{a}_{i-1}(r_{a_{i-1}}), \\ \tilde{W}_{ax}^*(r_{W_{ax}}) &\neq \tilde{W}_{ax}(r_{W_{ax}}), \tilde{W}_{aa}^*(r_{W_{aa}}) \neq \tilde{W}_{aa}(r_{W_{aa}}), \\ \tilde{b}_a^*(r_{b_a}) &\neq \tilde{b}_a(r_{b_a}) \end{aligned}$$

occurs. As x_i is public, a mismatch in $\tilde{x}_i$ is detected directly. The remaining mismatches concern committed polynomials $a_{i-1}, W_{ax}, W_{aa}, b_a$ and proceed to Phase 4. Case (ii) yields a mismatch on AUX_i at point $r_{\text{AUX},2}$, which will be proceed to Phase 4.

Phase 3: Phase 3 invokes the bit-decomposition sumchecks of Equations (3) and (4) on AUX_i . If $\mathcal{P}^*$ provides any non-binary bits or inconsistent reconstruction, then by the soundness of these sumchecks and the Schwartz–Zippel lemma, with high probability we obtain

$$\widetilde{\text{AUX}}_i^*(r_{\text{AUX},3}) \neq \widetilde{\text{AUX}}_i(r_{\text{AUX},3}),$$

i.e., an inconsistent evaluation claim for the committed auxiliary polynomial at point $r_{\text{AUX},3}$, which is handled in Phase 4.

Phase 4: We now collect all inconsistent evaluation claims. From Phases 1 and 2 we may have mismatches on committed polynomials among

$$a_i, a_{i-1}, W_{ax}, W_{aa}, W_{ya}, b_a, b_y,$$

each at a single challenge point.

For the auxiliary polynomial AUX_i , we may have up to three claimed evaluations at points $r_{\text{AUX},1}, r_{\text{AUX},2}, r_{\text{AUX},3}$ arising from Phases 1, 2, and 3. The parties apply the evaluation reduction sumcheck to aggregate these claims into a single evaluation $\widetilde{\text{AUX}}_i(r_{\text{in},\text{AUX}})$. By the soundness of the evaluation reduction sumcheck and the Schwartz–Zippel lemma, if any of the original evaluation claims for AUX_i is incorrect, then with high probability we have

$$\widetilde{\text{AUX}}_i^*(r_{\text{in},\text{AUX}}) \neq \widetilde{\text{AUX}}_i(r_{\text{in},\text{AUX}}).$$

Finally, the verifier checks all claimed evaluations against the corresponding commitments by calling PCS.Open on $\sigma_{a_i}, \sigma_{a_{i-1}}, \sigma_W, \sigma_{\text{AUX}_i}$ at their respective challenge points. By the soundness and binding of PCS, it is infeasible for a prover to open a commitment to an incorrect evaluation. Hence any of the mismatches identified above cause $\mathcal{V}$ to reject except with negligible probability.

Knowledge soundness follows immediately from the above. As PCS is knowledge sound, the extractor of Protocol A can invoke the extractor of PCS to obtain the witnesses committed to by the prover in $\sigma_W, \sigma_{a_{i-1}}, \sigma_{a_i}$, and σ_{AUX_i} . As Protocol A is sound, these witnesses satisfy the RNN step relations. Moreover, since GKR, SumMatMul, and PCS are zero-knowledge, the simulator of Protocol A can invoke the simulators of these building blocks to generate proof messages and commitments indistinguishable from a real protocol execution.

2) *Proof of Theorem III.2:*

a) *Correctness.*: Suppose the prover honestly executes each RNN step i on inputs (x_i, a_{i-1}) using model W , and runs A.PROVE with the correct witnesses. By completeness of GKR, SumMatMul, and PCS, these per-step proofs and commitments are valid. Since leaf packages are satisfying, and the augmented aggregation function F_A only verifies existing proofs and re-applies complete aggregation and sum-check procedures, every internal-node proof produced by PROVEINTERNALNODE is also satisfying. By induction over the recursion tree, the root proof package is satisfying. Finally, VERIFY recomputes Fiat–Shamir challenges, verifies the root GKR/sumcheck circuit, checks the wiring aggregate, validates the aggregated data instance with AGG.V, compares the aggregated public inputs with the locally computed aggregate, and runs PCS.Verify on the final aggregate opening. Each of these steps succeeds with probability 1 for an honest prover. Therefore, every honestly generated proof π_{zip} is accepted by the verifier.

b) *Knowledge Soundness.*: We work in the Fiat–Shamir random oracle model. Let ξ_{AGG} be the extractor of AGG, and let ξ_{step} be the extractor for ProofOfRNNStep. Suppose an adversary returns a satisfying tuple $(i, \sigma_W, \sigma_{a_0}, \sigma_{a_i}, \pi_{\text{zip}}, pp)$ such that

$$\text{Verify}(i, \sigma_W, \sigma_{a_0}, \sigma_{a_i}, \pi_{\text{zip}}, pp) = 1.$$

We construct an extractor ξ that outputs a valid RNN inference trace.

First, ξ decompresses π_{zip} to obtain the root proof package $\pi_{\text{root}} = (\pi_G, \pi_A, \pi_E, \{\text{agg}_{\text{in},m}\}_{m=1}^k, \text{agg}_{\text{root}}, \{\text{wp}_{\text{in},m}\}_{m=1}^k, \text{wp}_{\text{root}})$. By soundness of the GKR/sumcheck verifier and of AGG.V, acceptance of π_{root} implies that agg_{root} and π_A certify a valid aggregation of all child instances, and that the augmented circuit F_A accepts all embedded child proofs. Since the root includes a final opening π_E for agg_{root} , ξ runs ξ_{AGG} on $(\text{agg}_{\text{root}}, \pi_A, \pi_E)$ to recover the underlying evaluation tuples, including the polynomial encoding of

$$u_{\text{root}} = \sigma_{a_0} \parallel \sigma_{a_{\text{child}_1}} \parallel \dots \parallel \sigma_{a_{\text{child}_k}} \parallel \pi_{\text{child}_1} \parallel \dots \parallel \pi_{\text{child}_k},$$

as committed to by $\sigma_{u_{\text{root}}}$. From u_{root} , ξ reads off the boundary state commitments $\{\sigma_{a_{\text{child}_j}}\}_{j=1}^k$ and the child proof packages $\{\pi_{\text{child}_j}\}_{j=1}^k$ for the next recursion level.

For each internal node, ξ applies the same procedure recursively: given a parent package π_{parent} with aggregate $(\text{agg}_{\text{parent}}, \pi_{\text{parent},A}, \pi_{\text{parent},E})$, ξ invokes ξ_{AGG} to extract the corresponding vector

$$u_{\text{parent}} = \sigma_{a_0} \parallel \sigma_{a_{\text{child}_1}} \parallel \dots \parallel \sigma_{a_{\text{child}_k}} \parallel \pi_{\text{child}_1} \parallel \dots \parallel \pi_{\text{child}_k}$$

and thus obtains the children’s state commitments and proof packages. By soundness of GKR, SumMatMul, PCS, and AGG, all extracted child packages are valid instances of F_A and their aggregates faithfully represent the underlying commitments and evaluations. Applying this recursively over the entire recursion tree reaches all leaf step proofs.

At each leaf step, ξ now has the step transcript $\pi_{i,G}$ and the associated commitments $(\sigma_W, \sigma_{a_{i-1}}, \sigma_{a_i})$. By running the step extractor ξ_{step} , ξ extracts concrete witnesses

$$(a_{i-1}, a_i, W_{ax}, W_{aa}, W_{ya}, b_a, b_y)$$

such that $(a_i, y_i) = \text{RNN}(x_i, a_{i-1}, W_{ax}, W_{aa}, W_{ya}, b_a, b_y)$.

Since PCS is binding and extractable, and all aggregations are certified by AGG, the extractor ξ outputs a globally consistent model $W = (W_{ax}, W_{aa}, W_{ya}, b_a, b_y)$ together with a sequence of hidden states $\{a_0, a_1, \dots, a_i\}$ that realize a valid RNN inference trace. Hence from any accepting proof, ξ can efficiently extract a full, correct RNN inference trace, establishing knowledge soundness of Protocol B.

c) Zero-knowledge. We prove zero-knowledge with respect to the final verifier. The simulator S samples a random model W^* and a random initial hidden state a_0^* , and commits to them via PCS to obtain σ_W^* and $\sigma_{a_0}^*$. With access to the RNN circuit and to the step-proof simulator for ProofOfRNNStep, S executes the prover-side of Protocol B for i iterations, using only simulated step proofs, simulated aggregation proofs from AGG, and simulated GKR/sumcheck transcripts at internal nodes. At the root, S invokes the PCS simulator to produce the final batched opening π_E^* for the aggregated commitment. Since PCS is hiding and admit a simulator, and ProofOfRNNStep, AGG, GKR, and the sumcheck protocol are zero-knowledge, all commitments and all proof messages in the simulated transcript are indistinguishable from those in a real execution. Therefore, the final simulated proof π_{zip}^* is computationally indistinguishable from a real proof produced by an honest prover.